Berlin University of Applied Sciences and Technology

Department VI – Computer Science and Media

Master's Thesis

by

Khaled Rafei

Student ID: 930677

in partial fulfilment of the requirements
for the degree of

Master of Engineering (M. Eng.)

in the study programme

Computer Engineering

Topic:

Data-Efficient Personalization for Webcam-Based Eye Tracking Using Pretrained Vision Models

Supervisor (BHT):	Prof. Dr. Kristian Hildebrand
External supervisor:	Prof. Dr. Meysam Shamsi
Examiner:	Prof. Dr. Robert Strzebkowski

Submitted:	5 November 2026

Abstract

Webcam-based eye tracking is inexpensive and available on almost every laptop. With generic models, however, it is often inaccurate, because people, cameras and environments differ strongly. This thesis investigates how a pretrained vision model can be adapted to a single person with only a small amount of calibration data.

A ResNet-18 pretrained on ImageNet serves as a frozen feature extractor. Only a small regression head is trained: first on the GazeCapture dataset (base model) and then on own webcam recordings from 18 sessions of one person (personalization). Accuracy is measured with the diagonal error, i.e. the error in percent of the screen diagonal.

On the own recordings, the base model is not better than a constant prediction of the screen centre (30.6 % versus 30.3 %); it therefore mainly serves as a starting point. With a realistic, temporal split of the data (training on the beginning, test on the end of a recording), personalization reduces the error from 31.8 % to 27.9 %, i.e. by about 12 %. In sessions with good recording conditions the gain is much larger (46 to 47 %) and also holds for a new recording. With a random split of the frames the error drops to 19.8 % (about one third). This value is too optimistic, however, because almost identical neighbouring frames end up in training and test. One training run takes only a few seconds on a CPU. Background and lighting have a larger influence than the design of the calibration. Removing wrongly labelled frames after target jumps improves the result only slightly.

The results show that data-efficient personalization is possible with simple means. Its benefit, however, must be assessed with a realistic data split. All recordings come from one person, so the results hold for this person only at first.

Keywords: eye tracking, gaze estimation, webcam, personalization, transfer learning, ResNet-18, GazeCapture

Contents

Abstract I

List of Figures V

List of Tables VI

List of Abbreviations VII

1. Introduction 1
- 1.1. Motivation . . . 1
- 1.2. Problem Statement and Goal . . . 1
- 1.3. Research Questions . . . 1
- 1.4. Contributions . . . 2
- 1.5. Structure of the Thesis . . . 2

2. State of the Art 3
- 2.1. Basics of Appearance-Based Gaze Estimation . . . 3
  - 2.1.1. Classical Methods . . . 3
  - 2.1.2. First Deep Learning Methods . . . 3
  - 2.1.3. Types of Methods . . . 4
- 2.2. Webcam-Based Eye Tracking: Everyday Use versus Laboratory . . . 5
- 2.3. Large Datasets for Gaze Estimation . . . 6
- 2.4. Network Architectures for Gaze Regression . . . 7
  - 2.4.1. Convolutional Neural Networks . . . 7
  - 2.4.2. Vision Transformers . . . 7
  - 2.4.3. Pretrained Foundation Models . . . 8
- 2.5. Pretrained Models for Gaze Estimation . . . 8
  - 2.5.1. Frozen Backbone or Full Fine-Tuning . . . 8
  - 2.5.2. Large-Scale MAE Pretraining: UniGaze . . . 9
  - 2.5.3. Image-Text Models: CLIP-Gaze and GazeCLIP . . . 11
- 2.6. Data-Efficient Personalization . . . 12
  - 2.6.1. Calibration Methods . . . 12
  - 2.6.2. Few-Shot Learning and Meta-Learning: FAZE . . . 13
  - 2.6.3. Parameter-Efficient Fine-Tuning (PEFT) . . . 14

2.7. Domain Shift, Generalization and Evaluation Protocols . . . 16
2.7.1. Evaluation with Unknown Persons . . . 16
2.7.2. Domain Adaptation and Domain Generalization . . . 16
2.7.3. Gaze Estimation for Several Persons . . . 17
2.7.4. Differences in Devices and Environment . . . 17
2.8. Test-Time Adaptation . . . 18
2.8.1. TTP-GE: Self-Supervised and Weakly Supervised Adaptation . . . 18
2.8.2. Other Label-Saving Methods . . . 18
2.9. Data Quality, Wrong Labels and Timing . . . 20
2.9.1. Data Quality in Eye Tracking . . . 20
2.9.2. Reaction Time and Saccades . . . 21
2.9.3. Filtering During Recording . . . 21
2.9.4. Filtering After Recording and Robust Learning . . . 21
2.10. Comparison and Critical Discussion . . . 23
2.11. Research Gaps and Link to This Thesis . . . 24
2.11.1. Research Gaps . . . 24
2.11.2. What This Thesis Contributes . . . 25

3. Methodology **26**
3.1. Datasets . . . 26
3.2. Recording the Personalization Data . . . 26
3.3. Pre-Processing . . . 27
3.4. Model and Training . . . 27
3.5. Evaluation . . . 28
3.6. Implementation . . . 28

4. Experiments and Results **30**
4.1. Experimental Setup . . . 30
4.1.1. Data and Splits . . . 30
4.1.2. Error Measure . . . 30
4.1.3. Procedure . . . 31
4.1.4. Normalization of the Targets . . . 32
4.1.5. Baselines . . . 34
4.2. Training Settings . . . 35
4.2.1. Batch Size . . . 35
4.2.2. Learning Rate . . . 35
4.2.3. Trainable Layers . . . 37
4.2.4. Output Activation . . . 38
4.2.5. Amount of Data . . . 39
4.2.6. Type of Split . . . 39

4.3. Spatial Error Analysis and Loss Functions . . . 40
4.3.1. Screen Partition . . . 40
4.3.2. Error Heat Maps . . . 40
4.3.3. Loss Functions . . . 41
4.3.4. Results of the Loss Comparison . . . 42
4.4. Personalization Sessions . . . 44
4.4.1. Setup . . . 44
4.4.2. Results of All Sessions (E1) . . . 44
4.4.3. Training Time of a Session . . . 47
4.4.4. Label Cleaning (Gen 4, E1b) . . . 47
4.4.5. Recording Conditions (Gen 4) . . . 48
4.5. Strategies for Splitting the Data (E2) . . . 49
4.6. Design of the Calibration Session . . . 51
4.6.1. Screen Coverage and Transition Frames . . . 51
4.6.2. Personalization Result (E3) . . . 52
4.7. Discussion of the Results . . . 53
4.8. Summary of the Results . . . 55

5. Discussion **57**
5.1. Answers to the Research Questions . . . 57
5.2. Relation to the State of the Art . . . 58
5.3. Limitations . . . 58

6. Conclusion and Outlook **60**
6.1. Conclusion . . . 60
6.2. Outlook . . . 61

References **62**

A. Appendix **67**
A.1. Commands to Repeat the Experiments . . . 67

Statutory Declaration **69**

List of Figures

4.1. Raw and normalized gaze targets compared. 33
4.2. Test error of single runs with four learning rates 36
4.3. Spatial error of the default model 41
4.4. Test error of the 18 sessions after personalization with both base models, compared with the constant baseline and the unchanged base model (`norm_subject`) on the same test frames (data from Table 4.13). 46

List of Tables

2.1. Overview of selected gaze estimation datasets. 7
2.2. Comparison of UniGaze and this thesis. 10
2.3. Comparison of CLIP-Gaze, GazeCLIP and this thesis. 12
2.4. Comparison of FAZE and this thesis. 15
2.5. Comparison of TTP-GE and this thesis. 19
2.6. Comparison of filtering after recording (DMRF) and phase logging. 22
2.7. Comparison of the most important works 23

4.1. Default setting of `exp.py` 31
4.2. Baseline errors E_{diag} in % 34
4.3. Batch size 35
4.4. Learning rate 35
4.5. Trainable parameters of the three variants (ResNet-18 with regression head, 11 603 650 parameters in total). 37
4.6. Trainable layers 37
4.7. Output activation 38
4.8. Amount of data (training/test) 39
4.9. Type of split 39
4.10. Comparison of L1 and sensitive L1 loss 42
4.11. Additional experiment: spatial error of four loss functions 43
4.12. Recording conditions of the 18 sessions 45
4.13. Personalization of the 18 sessions with two base models 45
4.14. Label cleaning in Gen 4 48
4.15. Strategies for splitting the data 50
4.16. Coverage and transition frames per design 51
4.17. Test error per design with random and sequential split 52
4.18. Test error per screen region 53
4.19. Summary of the results and the chosen setting 56

List of Abbreviations

Abbreviation	Meaning
AdamW	Adam optimizer with decoupled weight decay
CI	Confidence interval
CLIP	Contrastive Language-Image Pre-training
CNN	Convolutional neural network
CPU	Central processing unit
DA	Domain adaptation
DG	Domain generalization
DINOv2	Self-Distillation with No Labels, version 2
FC	Fully connected (layer)
fps	Frames per second
L1	Loss based on the mean absolute error
LoRA	Low-Rank Adaptation
MAE	Masked autoencoder
MAML	Model-Agnostic Meta-Learning
MLP	Multi-layer perceptron
PEFT	Parameter-efficient fine-tuning
PoR	Point of regard (gaze point on the screen)
ReLU	Rectified linear unit
RMSE	Root mean squared error
SD	Standard deviation
SVR	Support vector regression
TTA	Test-time adaptation
ViT	Vision Transformer
Symbols	
E_{diag}	Diagonal error: Euclidean RMSE in percent of the screen diagonal

1. Introduction

1.1. Motivation

Eye tracking measures where a person is looking. This information is useful for many applications: for controlling a computer without hands, for example for people with physical disabilities, for usability studies of websites, for learning platforms, and for research in psychology and medicine. Professional eye trackers work with infrared light and special cameras. They are very accurate, but expensive and not available everywhere.

An ordinary webcam, in contrast, is built into almost every laptop. If the gaze could be estimated with a webcam, eye tracking would be possible for very many people without extra hardware. But this is difficult. The images have a low resolution, and the eyes are only a few pixels large. The light changes, the head moves, many people wear glasses, and every person has a different eye and face shape. A generic model trained on many persons therefore often fits a new person badly.

1.2. Problem Statement and Goal

One solution is *personalization*: one takes a pretrained model and adapts it to one person with a little data. For this, the person looks for a short time at a point that moves across the screen (calibration). The calibration should be short, however, because nobody wants to calibrate for several minutes before every use. Personalization must therefore be *data-efficient*.

The goal of this thesis is to study how well a pretrained vision model can be adapted to one person with a short webcam calibration. The model is a ResNet-18 pretrained on ImageNet. Its backbone stays frozen, and only a small regression head is trained. The head is first trained on the GazeCapture dataset and then personalized with own recordings.

1.3. Research Questions

The thesis answers four research questions (RQ):

RQ1 How much does a short personalization lower the error compared with a generic model and compared with a simple estimate that always gives the same point?

RQ2 Which training setting is suitable for the regression head (trainable layers, activation, loss function, learning rate, batch size)?

RQ3 How do the data split (random, temporal, two videos) and the design of the calibration (movement of the target, removal of transition frames) affect the result?

RQ4 What is the influence of the recording conditions (light, background, glasses, length, device or webcam)?

1.4. Contributions

The thesis makes the following contributions:

- **Own recordings:** 27 calibration sessions with two laptop webcams, 18 of them evaluated. All recordings come from one person (the author). The sessions differ in the movement of the target, light, background, glasses and length. The newest generation stores the phase of the target for every frame (`WAIT` or `MOVE`).
- **Reproducible pipeline:** fixed seeds, frozen backbone, stored features (feature cache) and stored predictions. One personalization run takes about two seconds on a CPU.
- **Corrected error measure:** a bug in the evaluation code was found and fixed. Baselines are always computed on the same data as the model.
- **Systematic study:** hyperparameters with five repeats and confidence intervals, a spatial analysis of the error, and a comparison of data splits and calibration designs.
- **Web demo:** a demo in the browser shows personalization live: calibration, gaze tracking and evaluation (Chapter 6).

1.5. Structure of the Thesis

Chapter 2 reviews the state of the art. Chapter 3 explains the data, the model and the procedure. Chapter 4 shows the experiments and results. Chapter 5 answers the research questions and names the limitations. Chapter 6 summarizes the thesis and gives an outlook.

2. State of the Art

This chapter gives an overview of gaze estimation with ordinary RGB cameras. First, I explain the basics (Section 2.1) and the special problems of webcams (Section 2.2). Then I present important datasets (Section 2.3), network architectures (Section 2.4) and pretrained models (Section 2.5). After that, the chapter covers personalization, domain shift, test-time adaptation and data quality. At the end, I derive the research gaps that this thesis addresses.

2.1. Basics of Appearance-Based Gaze Estimation

Eye tracking uses images of the eyes or the face to find out where a person is looking. There are two kinds of output: the three-dimensional gaze direction (usually two angles, pitch and yaw) or the two-dimensional *point of regard* (PoR). The PoR is the point on the screen that the person looks at. This thesis estimates the 2D PoR from images of an ordinary webcam.

2.1.1. Classical Methods

Appearance-based methods learn a direct mapping from the image to the gaze target. They do not need a geometric model of the eye. Early methods used hand-crafted image features, for example Local Binary Patterns (LBP), Histograms of Oriented Gradients (HoG) or SIFT. These features were combined with simple regressors such as linear regression, random forests or support vector regression (SVR). Such methods are fast. With head movement and changing light, however, they become much less accurate, because the features do not separate these changes well from the gaze information.

2.1.2. First Deep Learning Methods

Convolutional neural networks (CNNs) can map face and eye images directly to screen positions. Krafka et al. [1] introduced *iTracker*, one of the first CNNs for mobile eye tracking without extra hardware. The model processes four inputs in separate branches: the left eye, the right eye, the face and a binary *face grid* (where the face is in the image).

Then the features are combined. Without calibration, the error is 1.71 cm on smartphones and 2.53 cm on tablets. For personalization, the authors train one SVR per user on the features of layer `fc1`. They use recordings of up to 13 fixed calibration points. With all 13 points, the error drops to 1.34 cm and 2.12 cm. With very few points, however, the SVR easily runs into *overfitting*: the model almost memorizes the few training samples and therefore becomes worse on new images. The model runs in real time on mobile devices.

An important contribution of this work is the GazeCapture dataset (Section 2.3). It was collected by *crowdsourcing* on smartphones and tablets in everyday conditions ("in the wild"): many volunteers recorded the data themselves with an app on their own device. It is still used as a benchmark and as a source for pretraining.

Relevance for this thesis. iTracker introduced three ideas that are still important: (1) the GazeCapture dataset, (2) personalization with calibration data and (3) combining features from several image regions. This thesis uses the same data. Instead of AlexNet and SVR, however, it uses a ResNet-18 [2] with a regression head and L1 loss. Here the regression head is a multi-layer perceptron (MLP), a small network of fully connected layers that computes the screen coordinates (x, y) from the image features of the ResNet-18. It also evaluates the base models with a subject-disjoint split (`norm_subject`) and not only with a random split (`norm_random`). Wrong labels are handled already during recording: the recording program stores the phase of the target for every frame (`WAIT` or `MOVE`).

2.1.3. Types of Methods

Gaze estimation methods can be divided into three classes:

1. **Model-based (geometric) methods** use a 3D model of the eyeball, pupil ellipses and light reflections on the cornea. They usually need high-resolution images and infrared light. In normal daylight they often fail.
2. **Feature-based methods** find geometric points in the face, for example eye corners or the centre of the iris. From these points they compute the gaze.
3. **Appearance-based methods** use the eye or face image itself as input. A deep neural network directly predicts 2D coordinates or 3D angles.

Thanks to progress in deep learning, appearance-based methods are now the standard under *unconstrained conditions*, that is, with free head pose, changing light and any distance to the camera.

Cheng et al. [3] also divide *remote-camera* methods (the camera stands in front of the person and is not worn on the head) into three groups: methods with a reconstructed 3D eye model (the eyeball is modelled geometrically, and the gaze is computed from the pupil

and light reflections), regression on 2D eye features, and appearance-based methods. They sort deep learning approaches by four aspects: feature extraction, network architecture, personal calibration, and devices and platforms. An important contribution is a benchmark. It unifies the pre-processing (face and eye detection, data normalization) and the post-processing (conversion between 2D gaze point and 3D gaze vector). This allows a fair comparison of methods. According to the benchmark, appearance-based methods reach about 5° at short distance or with high resolution. At large distance and low resolution (for example Gaze360 and RT-GENE), the errors are much larger. As open topics, the authors name: (1) more robust features that do not depend on the environment, (2) high accuracy with fast and easy calibration – here there is a trade-off between calibration time and accuracy – and (3) the explainability of the learned features.

Other surveys give additional background. Liu et al. [4] describe the path from classical methods to deep learning; I use this work mainly as a historical reference. Pathirana et al. [5] cover deep learning methods for single and multiple users. They compare CNN architectures, some of which are pretrained on ImageNet. Newer pretrained models such as DINOv2, CLIP or MAE are not the focus of these surveys. For benchmark comparisons I therefore mainly use Cheng et al. [3].

2.2. Webcam-Based Eye Tracking: Everyday Use versus Laboratory

Webcams are cheap and almost everywhere: in laptops, on desktop computers and in smartphones. No extra hardware is needed. But the conditions are much harder than for special eye trackers with infrared light:

- **Infrared systems** use high-resolution cameras, infrared light and reflections on the cornea. Liu et al. [4] report typical accuracies of 0.5° to 2° for such methods. Commercial devices work at fixed distances (for example 50 cm to 80 cm) and reach 0.3° to 0.5° under optimal conditions.
- **Webcams in everyday use** deliver low-resolution images without controlled light. The ambient light changes, the position of camera and screen is different on every device, and distance and head pose are free. Liu et al. [4] report typical errors above 3° for appearance-based methods. The benchmark of Cheng et al. [3] shows about 5° at short distance and much more at large distance and low resolution.

Two well-known works still show that webcam eye tracking is useful in practice. Papoutsaki et al. [6] present *WebGazer*, an open-source eye tracker that runs completely in the browser. WebGazer does not need its own calibration. With *ridge regression*, it continuously learns from mouse clicks and mouse movements which eye features belong to which screen position.

Ridge regression is a linear regression with a penalty on large weights (L2 regularization); this keeps the model stable even with few and noisy data. The accuracy is enough to find the rough area a person looks at, but not for detailed analysis. Valliappan et al. [7], in contrast, pretrain a CNN on GazeCapture, fine-tune it with calibration data and add a small regression model per person. On smartphones at 25 cm to 40 cm distance they reach 0.6° to 1°. About 30 seconds of calibration reduce the error from 1.92 cm to 0.46 cm. According to the authors, this is as accurate as much more expensive mobile eye trackers. The conditions, however, were favourable (head almost frontal, short distance). Both works show: without special hardware, personalization is the most important factor for accuracy.

Properties of the person also affect the accuracy, for example eye shape, face shape, glasses and the usual head pose. Generic models therefore often have a systematic offset for new persons. This is why personalization is a central research topic (Section 2.6). This thesis studies exactly such conditions, among others two devices with different webcams, glasses and backgrounds.

2.3. Large Datasets for Gaze Estimation

Modern appearance-based methods need large and varied datasets. Table 2.1 summarizes the datasets that are important for this thesis.

- **MPIIGaze** [8] is one of the first standardized everyday datasets. It contains 213 659 face images of 15 persons. They were recorded over several months during normal work on a laptop [9]. The variant MPIIFaceGaze with face images is usually evaluated with leave-one-subject-out (each person is the test person once).
- **GazeCapture** [1] contains 2 445 504 images of 1474 persons, recorded by crowdsourcing on iPhones and iPads. Light and head pose vary a lot. GazeCapture is the data basis of the base models in this thesis. Reasons for this choice: it is one of the largest and most diverse datasets (many persons, devices and environments), and its labels are 2D points on the screen (point of regard) – exactly the quantity this thesis estimates.
- **ETH-XGaze** [10] is a high-resolution dataset with several cameras. It contains more than one million images of 110 persons with very large head and gaze angles.
- **EVE** [11] contains synchronized screen content, camera images and gaze data. 54 persons were recorded with four cameras, more than 12 million images in total.
- **Gaze360** [12] contains indoor and outdoor recordings of 238 persons. The 3D gaze labels cover a very wide range of head poses and distances (about 172 000 images in training, validation and test).

- **EyeDiap** and **RT-GENE** are mainly used for tests across datasets. RT-GENE contains, among others, larger camera distances and head-mounted devices.

Table 2.1.: Overview of selected gaze estimation datasets.

Dataset	Size	Persons	Recording conditions
MPIIGaze	213 659 images	15	Laptop webcam, everyday use over months
GazeCapture	2 445 504 images	1474	Smartphones and tablets, crowdsourcing
ETH-XGaze	¿1 million images	110	Lab with several cameras, extreme head poses
EVE	¿12 million images	54	Screen and cameras, video sequences
Gaze360	approx. 172 000 images	238	Indoor and outdoor, many head poses and distances

2.4. Network Architectures for Gaze Regression

2.4.1. Convolutional Neural Networks

CNNs such as ResNet-18/50 [2], VGG or MobileNet are the usual feature extractors for eye and face images. Their local convolutions detect fine textures and edges in the eye region. Special gaze models are also often based on CNNs: FAZE [13], for example, uses a DenseNet-based encoder-decoder to separate gaze, head pose and appearance. This thesis uses ResNet-18.

2.4.2. Vision Transformers

Vision Transformers (ViTs) [14] split an image into small patches. With self-attention they capture relations across the whole image, for example between the face, the head rotation and the eyes. UniGaze [15] shows that ViTs pretrained with MAE (sizes Base, Large and Huge) are very good backbones for gaze estimation. TTP-GE [16], in contrast, combines two ResNet-18 branches with a transformer fusion. GazeCLIP [17] shows that transformers are not always better: there, a CLIP-pretrained ResNet-50 reaches 3.5° on MPIIFaceGaze, while the ViT-B/32 variant reaches only 3.7°. Ye et al. [18], on the other hand, show with *TransGaze* that even plain ViTs with large-scale pretraining achieve very good results on GazeCapture and MPIIFaceGaze. Self-supervised DINO pretraining worked best there. One should therefore test CNNs and transformers equally and not assume that one architecture is generally better.

2.4.3. Pretrained Foundation Models

In transfer learning, one uses a model that was pretrained on a very large number of images. Its features are useful in general. One can freeze the backbone and train only a small regression head on top. Four kinds of pretraining are important for this thesis:

- **Supervised pretraining on ImageNet**: the model learns with labels given by humans (here: object classes). This is the classical starting point for CNNs such as ResNet [2]. This thesis uses such a model.
- **DINOv2** [19]: learns robust features with *self-supervised learning*: the learning target comes from the images themselves, without manual labels.
- **CLIP** [20]: learns image and text together by contrastive learning on image-text pairs.
- **Masked autoencoders (MAE)** [21]: learn by reconstructing hidden parts of the image.

The next section describes how these features are used for gaze estimation.

2.5. Pretrained Models for Gaze Estimation

2.5.1. Frozen Backbone or Full Fine-Tuning

There are two basic strategies to adapt a pretrained model to gaze estimation. In *full fine-tuning*, all weights are trained. The model adapts most strongly, but it needs a lot of data. With little data there is a risk of overfitting or *catastrophic forgetting*: the model loses what it learned during pretraining. With a *frozen backbone* (for example ResNet-18 or ViT-B/14), the pretrained weights stay fixed. Only a small regression head is trained. This gives far fewer trainable parameters, and the general features are kept. This is important when only little calibration data is available.

Kumar et al. [22] explain this trade-off in theory and with experiments. They compare full fine-tuning with *linear probing*, where only the last linear layer is trained. On ten datasets with distribution shift, fine-tuning is on average 2 % better on data like the training data, but 7 % worse on shifted data. The reason: when head and backbone are trained at the same time and the head is still random at the start, the pretrained features get distorted. The authors therefore propose LP-FT: first linear probing, then fine-tuning. This is directly relevant for personalization. The calibration data of one person are few and cover the later use only partly. A frozen backbone can protect the generalization here.

The regression head is usually a multi-layer perceptron (MLP). It maps the features, for example the 512 numbers of a ResNet-18, to the coordinates (x, y). Important choices are the output activation (for example sigmoid, Gaussian or ReLU) and the loss function (for example L1, smooth L1 or variants weighted by position). Between the two extremes there are parameter-efficient methods such as adapters or LoRA [23] (Section 2.6.3).

2.5.2. Large-Scale MAE Pretraining: UniGaze

Qin et al. [15] present UniGaze, a large pretraining approach made specially for gaze estimation. With masked autoencoders, the model learns from about 1.6 million normalized face images of more than 260 000 persons. The data come from real sources (VGGFace2, VFHQ, CelebV-Text) and from synthetic datasets (FaceSynthetics, SFHQ-T2I, FFHQ-NV, XGaze-Dense). A central result is how important data normalization is. Here, the face image and the gaze label are rotated and scaled into a common space. This removes a large part of the variation caused by head pose and distance. Zhang et al. [24] were the first to study this method in detail. A variant without a scaling factor improves the accuracy by 9.5 to 32.7 %, depending on the test. UniGaze confirms this: in the ablations (UniGaze-B, real pretraining data only, leave-one-dataset-out), normalization reduces the angular error on XTest from 14.56° to 11.95°. With additional synthetic data, 11.78° is reached.

UniGaze uses ViTs of the sizes Base, Large and Huge. During pretraining, 75 % of the image is hidden. ViT-Huge is pretrained for 300 epochs, which takes about 120 h on four NVIDIA H100 GPUs. Then the models are fully fine-tuned with L1 loss on MPIIFaceGaze, ETH-XGaze, EYEDIAP, Gaze360 and GazeCapture. The authors test three protocols:

- **Domain generalization** (training on ETH-XGaze or Gaze360, test on MPIIFaceGaze or the screen part of EYEDIAP): UniGaze-H reaches 5.57° (X→M), 4.65° (X→ECS), 5.65° (G360→M) and 5.86° (G360→ECS). This is better than other methods, for example CLIP-Gaze with 6.41°, 7.51°, 6.89° and 7.06°.
- **Leave-one-dataset-out** (training on four datasets, test on the fifth): UniGaze-H reaches 11.29° (XTest), 4.51° (MPIIFaceGaze), 5.47° (GazeCapture), 5.88° (EYEDIAP) and 12.37° (Gaze360).
- **Joint training on all datasets**: UniGaze-H reaches 4.46° (XTest), 5.08° (MPIIFaceGaze), 3.20° (GazeCapture), 5.16° (EYEDIAP) and 9.07° (Gaze360).

The ablations show: more pretraining data improve generalization, and real plus synthetic data are better than real data only. Normalization and many different head poses are also important. If the variety of head poses is limited, the error on XTest rises from 11.95° to 16.70°. Comparisons with DINO and FaRL suggest that the composition of the data can be more important than their amount.

Limitations. UniGaze leaves some questions open that matter for personalization with webcams. The results are based on complete datasets and full fine-tuning, not on a short calibration with few images of one person. Lightweight strategies such as head-only training, LoRA or adapters are not the focus. The effect of wrong labels in short calibrations is not studied either. Pretraining also needs a lot of computing power (ViT-Huge: about 120 h on 4×H100). It also remains open whether the improvements hold under real webcam conditions – with glasses, changing backgrounds, low resolution and different webcams.

Relevance for this thesis. UniGaze is an important reference for gaze-specific pretraining. UniGaze studies generalization across many persons. This thesis asks the complementary question: how well can a pretrained model be adapted to one single person with a short calibration (here 30 s to 120 s, that is, about 180 to 720 labels)? This thesis uses a ResNet-18 pretrained on ImageNet for this. UniGaze, DINOv2 and CLIP models are a natural next step (Table 2.2).

Table 2.2.: Comparison of UniGaze and this thesis.

Criterion	**UniGaze-H**	**UniGaze-B**	**This thesis**
Pretraining data	1.6 million face images, ¿260 000 persons	Same data	ImageNet (ResNet-18), regression head on GazeCapture
Backbone	ViT-Huge	ViT-Base	ResNet-18, frozen
Fine-tuning	Whole model	Whole model	Regression head only
Calibration data	Complete training datasets	Complete training datasets	30 s to 120 s per session (approx. 180–720 labels)
Evaluation	Across datasets, domain generalization	Across datasets, domain generalization	Subject-disjoint base models, personalized webcam sessions
Wrong labels	No phase-based filtering	No phase-based filtering	`WAIT`/`MOVE` logging
Compute	Approx. 120 h on 4×H100	Only given for ViT-H	No pretraining; personalization approx. 2 s on a CPU
Angular error (leave-one-dataset-out)	4.51°–12.37°	4.73°–12.41°	Diagonal error in %, not directly comparable

2.5.3. Image-Text Models: CLIP-Gaze and GazeCLIP

Two recent works use the image-text model CLIP [20] for gaze estimation, but with different goals.

CLIP-Gaze. Yin et al. [25] want to improve domain generalization with CLIP-Gaze. The model should stay good in a new environment without labelled data from that environment. The idea: factors that have nothing to do with gaze are described by texts, such as "An image of a face with {condition}." These factors form three groups: appearance (identity, facial expression, hair, age, etc.), accessories (glasses, hat, make-up, etc.) and image quality (noise, blur, light). Three losses guide the training: a distillation loss aligns the gaze features with the CLIP image encoder. A separation loss makes them different from the gaze-irrelevant features. A rank loss makes sure that similar gaze directions also have similar features. In addition, *Personalized Context Optimization* (PCO) adapts the texts to each person. CLIP stays frozen. A separate network of ResNet-18, an MLP filter and a regression head is trained. With ETH-XGaze and Gaze360 as source and MPIIFaceGaze and EyeDiap as target, CLIP-Gaze reaches about 6.97° on average over four tasks.

GazeCLIP. Wang et al. [17] use coarse gaze information from text for supervised gaze regression. Without training (zero-shot), a prompt module uses texts such as "A photo of a face gazing [class]" to find a coarse direction from {Front, Down, Left, Right}. The image encoder is a CLIP-pretrained ResNet-50 and is fine-tuned. The text encoder (a transformer with 12 layers) stays frozen. A cross-attention module connects both: the text is the query, and the image features are key and value. An MLP with three layers ($1024 \rightarrow 256 \rightarrow 128 \rightarrow 2$) predicts pitch and yaw. GazeCLIP reaches 3.5° on MPIIFaceGaze, 4.7° on EyeDiap and 7.3° on RT-GENE. Cross-attention (3.6°) is better than simple concatenation (4.2°) or addition (4.4°). A fine-tuned image encoder with a frozen text encoder (3.6°) is much better than two frozen encoders (8.9°). Errors mainly occur in bad light and when hair or glasses cover the eyes. In addition, the zero-shot classification can give the wrong direction in a new environment.

Limitations and relevance for this thesis. Both works show that CLIP features are useful for gaze estimation. They do not study personalization with a short calibration, however. For CLIP-Gaze, it stays open whether adapting the CLIP image encoder improves the estimate for one person. GazeCLIP needs full supervised training on benchmark data. It covers neither personalization with few images nor wrong calibration labels, and according to the publication its data are only available on request. CLIP features were not yet studied in this thesis; they are a candidate for the outlook (Table 2.3). The angular errors of these works are only reference values. They cannot be compared directly with the diagonal error of this thesis.

Table 2.3.: Comparison of CLIP-Gaze, GazeCLIP and this thesis.

Aspect	**CLIP-Gaze [25]**	**GazeCLIP [17]**	**This thesis**
Goal	Domain generalization	Supervised gaze regression	Personalization with webcam and little data
Use of CLIP	Image and text encoder frozen	Image encoder fine-tuned, text encoder frozen	Not used (outlook)
Architecture	ResNet-18, MLP filter, regression head	CLIP ResNet-50, cross-attention, MLP	ResNet-18 (ImageNet), regression head
Labels of the target person	None	Complete training data	30 s to 120 s calibration
Reported error	Approx. 6.97° (mean of 4 tasks)	3.5° / 4.7° / 7.3°	Diagonal error in %
Data quality	Models gaze-irrelevant factors	Coarse direction from text (uncertain)	`WAIT` frames are removed

2.6. Data-Efficient Personalization

Generic models often have a systematic offset for new persons. It is caused by differences in the eye, the face shape and the head pose. Park et al. [13] speak of anatomical differences between persons that limit the accuracy of generic networks. Personalization tries to close this gap with as little calibration effort as possible. The following sections describe classical calibration, few-shot and meta-learning, and parameter-efficient methods.

2.6.1. Calibration Methods

In active calibration, the person looks at known points on the screen one after the other. The coordinates of these points are the labels for the adaptation. Typical are a few fixed points or a point that moves across the screen. iTracker [1] uses up to 13 fixed points and trains one SVR per user on them (Section 2.1.2). He et al. [26] show that good methods with only 2 to 5 points per person are better than earlier methods with more than 13 points. On smartphones, their best model with at most five points is 24 to 26 % more accurate than the state of the art at that time and runs in real time on the device. They also propose a method that needs only unlabelled face images. Valliappan et al. [7] use, in addition to points, a *zigzag calibration*: a target moves slowly for 60 seconds from top left to bottom right while the front camera records at 30 Hz. Already about 100 images reduce the error almost to one quarter.

The methods differ not only in *how much* data they need, but also in *which part* of the model they adapt:

- **Offset calibration:** Chen and Shi [27] split the gaze angle into a part that is the same for all persons and a personal offset (bias). They observe that the offset between persons is larger than the variation within one person. The calibration then needs only one gaze point. On MPIIGaze and EYEDIAP, the method is up to 10 % better than comparable models. Already about nine calibration images reduce the error clearly.

- **Few person parameters:** Lindén et al. [28] describe personal differences with the *Spatial Adaptive Gaze Estimator* (SPAZE) by a few latent parameters. Calibration is then only a small optimization problem and hardly prone to overfitting. With nine examples, SPAZE reaches 2.70° on MPIIGaze, 14 % better than the state of the art at that time. Three parameters per eye are enough to describe the personal differences well.

- **Differential estimation:** Liu et al. [29] train a CNN that predicts the *difference* in gaze between two eye images of the same person. Together with labelled reference images from the calibration, this gives the gaze in the new image. Because both images come from the same person, disturbances such as eye shape, eyelid closure or light partly cancel out. On EYEDIAP, MPIIGaze and UT-Multiview, the method beats the state of the art with only one calibration image.

- **Personalization without calibration:** Zhao et al. [30] propose the *Episode-based Personalization Network* (EbPN). A separate branch of the network should compensate personal deviations without calibration data.

Together, these works show: a large part of the personal error can be explained with a few adapted parameters. This supports the strategy of this thesis to adapt only a small regression head on a frozen backbone.

Cheng et al. [3] name a basic trade-off: longer calibrations make the estimate more accurate, but they are less convenient. High accuracy with a short and easy calibration is therefore an open research question. Another point is often overlooked: the quality of the calibration labels themselves. When a target jumps, the gaze follows only after a short delay. Frames from this transition can have a wrong label (Section 2.9).

2.6.2. Few-Shot Learning and Meta-Learning: FAZE

Park et al. [13] present FAZE (*Few-Shot Adaptive Gaze Estimation*), an influential method for personalization with few calibration images. It combines separated (disentangled) features for gaze, head and appearance with Model-Agnostic Meta-Learning (MAML). FAZE does not train a new model for every person. Instead, it learns a starting point from which a few examples are enough for a fast adaptation.

FAZE has two parts. A *Disentangling Transforming Encoder-Decoder* (DT-ED) based on DenseNet represents appearance, gaze direction and head pose in separate codes. Because these codes rotate together with the input (rotation-equivariant), changes of gaze and head can be described as rotations in feature space. Training uses image pairs of the same person with a reconstruction loss, a gaze loss and a consistency term. The second part is a gaze estimator that is meta-trained with MAML: over many training persons it learns a starting point from which a few gradient steps produce a personal model. This should avoid overfitting, which is a risk for normal fine-tuning with very little data.

FAZE is trained on GazeCapture and tested on GazeCapture and MPIIGaze. With $k = 9$ calibration images, the mean error on MPIIGaze is 3.14°. For all $1 \leq k \leq 256$, FAZE is better than the compared methods. On GazeCapture, the error drops by 19 % compared with the state of the art, from 3.91° to 3.18°.

Limitations and differences. FAZE shows that calibration depends not only on the amount of data, but also on the features learned before and on the type of adaptation. The features of FAZE, however, come from supervised training with gaze data and a DenseNet. General pretrained models (DINOv2, MAE, CLIP) are not studied. The adaptation uses meta-learning; simpler ways such as head-only training, LoRA or adapters are not the focus. Webcam-specific problems such as changing image quality, glasses, backgrounds and different cameras are not considered either, and neither is the removal of calibration frames by recording phase. This thesis takes up some of these points. Because FAZE reports angular errors and this thesis uses an error relative to the screen diagonal, the numbers are not directly comparable.

Yu et al. [31] extend adaptation with few images: from a few reference images of a person they create additional training images by gaze redirection and use them to adapt a generic model.

2.6.3. Parameter-Efficient Fine-Tuning (PEFT)

Parameter-efficient methods adapt only a small part of the parameters. The pretrained backbone stays almost unchanged. This saves memory and computing time and lowers the risk of overfitting when only little calibration data is available. Three variants are important for this thesis:

- **Head-only training (FC only):** The backbone stays completely frozen; only the regression head is trained. The backbone features can be computed once and stored. This makes the method especially fast. This thesis uses it.
- **Adapters** [32]: Small extra modules are inserted between the layers of a frozen network. Only they are trained.

Table 2.4.: Comparison of FAZE and this thesis.

Aspect	**FAZE [13]**	**This thesis**
Goal	Personalization of the gaze direction with few images	Data-efficient personalization of the screen position with a webcam
Features	Disentangled codes, DenseNet encoder-decoder	ResNet-18, pretrained on ImageNet and frozen
Pretraining	Supervised on GazeCapture	ImageNet; regression head on GazeCapture
Adaptation	MAML and gradient steps per person	Regression head only
Calibration data	$k = 1$ to 256 images	30 s to 120 s (approx. 180–720 labels)
Label quality	No phase-based filtering	`WAIT` frames removed via phase logging
Evaluation	GazeCapture and MPIIGaze	18 webcam sessions under changing conditions
Reported error	3.14° (MPIIGaze, $k = 9$)	Diagonal error in %, not directly comparable

- **Low-Rank Adaptation (LoRA)** [23]: The pretrained weight matrices W stay fixed. Only a change $\Delta W = BA$ made of two narrow matrices with low rank r is trained. This gives far fewer trainable parameters.

In gaze estimation, such methods have hardly been studied systematically. One exception is TPGaze [33]. It transfers prompt tuning from language processing: an extra prompt with less than 1 % of the parameters of a ResNet-18 is adapted at test time without labels. Meta-learning makes sure that this adaptation really lowers the gaze error (Section 2.8). Bao and Lu [34] also show that especially the last fully connected layer, which maps many image features to the gaze, overfits easily. So the design of the head matters even for head-only training. The other works discussed here use full fine-tuning (UniGaze, GazeCLIP), meta-learning (FAZE) or a frozen model that guides a separately trained network (CLIP-Gaze). This thesis studies head-only training for personalization with short webcam calibrations. Adapters and LoRA are left for future work.

2.7. Domain Shift, Generalization and Evaluation Protocols

2.7.1. Evaluation with Unknown Persons

How well a model generalizes to new persons depends strongly on the evaluation protocol. With a random split (`norm_random`), images of the same person can end up in training and test. In this thesis, groups of person and target point are split: all frames in which

a person looks at the same point are either in training or in test, but the same person appears in both parts. The model can then use features of this person, and generalization is overestimated. With a subject-disjoint split (`norm_subject`), the persons are split instead (here 80 % training, 20 % test): all frames of a person are only in training or only in test. Then this data leakage does not exist (similar to leave-one-subject-out). One then measures the performance on truly unknown persons. In the base experiments of this thesis on GazeCapture, `norm_subject` gives a test error of 28.39 % and `norm_random` 27.72 %. Most of this difference, however, comes from the different test data: the constant baseline is also lower on `norm_random` (27.66 % instead of 28.46 %, see Chapter 4).

Tests across datasets are a different case, for example leave-one-dataset-out in UniGaze [15]. There, a whole dataset is held back. The two protocols therefore measure different kinds of generalization: to unknown persons in the same environment, or to unknown recording conditions.

2.7.2. Domain Adaptation and Domain Generalization

When training and test data differ in persons, cameras or environment, appearance-based methods become much worse. There are two counter-strategies. Domain adaptation uses (usually unlabelled) data from the target environment. Domain generalization needs no target data at all.

CRGA. Wang et al. [35] propose *Contrastive Regression Gaze Adaptation* (CRGA), an unsupervised domain adaptation method. A module for *Contrastive Domain Generalization* (CDG) learns stable features in the source domain: features with similar gaze are pulled together, features with very different gaze are pushed apart. A module for *Contrastive Self-training Adaptation* (CSA) then learns with pseudo-labels in the target domain. With ETH-XGaze and Gaze360 as source and MPIIGaze, RT-GENE, GazeCapture and EyeDiap as target, the authors improve the baseline by 40.4 %, 34.7 %, 55.8 % and 34.3 % (source ETH-XGaze) and by 31.7 %, 30.5 %, 32.9 % and 23.8 % (source Gaze360).

CRGA therefore solves a different problem: adaptation to a new environment without labelled target data. This thesis, in contrast, uses a few labelled calibration data of the target person. This allows supervised training with real labels, not only with pseudo-labels. Self-training with pseudo-labels can also create a confirmation bias: wrong predictions are reinforced in further training. In addition, this thesis is about differences between persons and not about switching between benchmark datasets. CRGA is therefore related work on robustness, but not a directly comparable baseline.

Domain generalization. Methods such as PureGaze [36] or CLIP-Gaze [25] (Section 2.5.3) try to remove gaze-irrelevant factors from the features already during training. Then the model also works in new environments without target data. UniGaze [15] shows that large gaze-specific pretraining can beat these methods in tests across datasets (Section 2.5.2).

2.7.3. Gaze Estimation for Several Persons

Zhang et al. [37] present GazeOnce, a one-stage method. It estimates the gaze of all persons in an image in a single pass and stays real-time even with more than ten faces. Besides the gaze direction, it also predicts face boxes and facial landmarks. A self-supervised loss checks whether the estimated 3D direction fits its 2D projections. For training, the authors create the synthetic dataset MPSGaze: they insert faces from ETH-XGaze with known gaze into matching faces from WiderFace. According to the authors, GazeOnce is more accurate and faster than existing methods, especially with many faces.

GazeOnce therefore solves a different problem: the gaze of many persons in free scenes, not personalization for one webcam user. Methodically interesting is the consistency between the 3D direction and its 2D projection. MPSGaze also shows that synthetic training data can differ from real webcam images.

2.7.4. Differences in Devices and Environment

Besides differences between datasets, many factors of device and environment affect the accuracy: glasses and occlusions, background and light, image resolution and camera distance, and the position of camera and screen. The literature gives some hints:

- Cheng et al. [3] observe much larger errors at large distance and low resolution (Gaze360, RT-GENE).
- GazeCLIP [17] names bad light and occlusion by hair or glasses as typical failure cases.
- UniGaze [15] stresses how important many different head poses and a wide range of gaze labels are.
- CLIP-Gaze [25] explicitly models accessories and image quality as gaze-irrelevant factors.

Systematic studies of these factors for personalization with short webcam calibrations are still largely missing. This thesis therefore also looks at two devices with different webcams, glasses, background and calibration length across several sessions (see Chapter 4).

2.8. Test-Time Adaptation

In test-time adaptation (TTA), a fully trained model is adapted to the person or the environment only during use, usually with few or no labels. Unlike classical calibration (Section 2.6.1), the user should have to do as little as possible.

2.8.1. TTP-GE: Self-Supervised and Weakly Supervised Adaptation

Nagpure et al. [16] present TTP-GE (*Test-Time Personalized Gaze Estimation*), a two-stage method for gaze estimation in the car. It is based on GazeDPTR V2. This architecture has two branches: one for the original face image and one for a normalized face image. A transformer combines the features. With position information from a point projection, the model can also recognize gaze zones.

Meta-training uses two loops based on MAML. In the inner loop, a self-supervised consistency loss (in the style of BYOL++) adapts the model to new persons – without gaze labels. The outer loop optimizes gaze regression and zone recognition with labelled data. At test time there are two steps: first, the model adapts in a self-supervised way to unlabelled images of the person; this helps with changed image conditions. Then the zone recognition is adapted with a few zone labels (for example mirror, dashboard, windscreen). These labels are collected in a short setup phase.

On IVGaze (44 705 images of 125 persons), the combination reaches a mean error of 6.00°. The baseline is at 6.78°, *weakly supervised adaptation* alone (here with a few coarse zone labels) at 6.13° and *self-supervised adaptation* alone (without any labels) at 6.72°. The share of estimates with less than 6° error rises from 49.95 % to 58.2 %. So the two steps complement each other: self-supervision handles the image conditions, and the zone labels provide personal information.

2.8.2. Other Label-Saving Methods

Some recent works pursue similar goals with other means:

- **TPGaze** [33] adapts only a meta-learned prompt at test time. It has less than 1 % of the parameters of a ResNet-18. A simple symmetry loss is enough as a goal without labels. According to the authors, the adaptation is up to ten times faster than other methods. It is tested in four scenarios across datasets.
- **ELF-UA** [38] treats adaptation *without labels* as its own problem: a few unlabelled images of the person are enough. Meta-learning uses a bound from domain adaptation as the loss function.

- **PIGaze** [39] obtains calibration data in the car automatically: the gaze at known zones such as mirror and windscreen must fit together geometrically (self-calibration). A pretrained base model is a *frozen* feature extractor. On top, a small adapter learns the personal mapping. On IVGaze and LBW, PIGaze is better than fine-tuning.

Limitations and differences. These methods want to reduce calibration effort with unlabelled or weakly labelled data. They mostly estimate 3D gaze directions and, for TTP-GE and PIGaze, work in the car with fixed zones. Their adaptation (meta-training, online updates, EMA target networks) is much more complex than simple head training. General pretrained models such as DINOv2, CLIP or MAE are not studied as backbones. This thesis follows the complementary path of *supervised* personalization with screen coordinates from a short calibration. It is interesting, however, that PIGaze – like this thesis – combines a frozen backbone with a small, personally trained module.

Table 2.5.: Comparison of TTP-GE and this thesis.

Aspect	**TTP-GE [16]**	**This thesis**
Approach	Test-time adaptation with unlabelled images and a few zone labels	Supervised personalization with labelled calibration data
Backbone	GazeDPTR V2: two ResNet-18 branches with transformer fusion	ResNet-18 (ImageNet), frozen
Labels	Unlabelled images and nine gaze zones	Screen coordinates; `WAIT` frames are removed
Adaptation	Meta-training, online updates, EMA target network	Regression head only
Application	Driver monitoring in the car	Webcam in front of a screen
Result	6.00° on IVGaze	Diagonal error in %, not directly comparable

2.9. Data Quality, Wrong Labels and Timing

Supervised personalization assumes that every calibration frame has the right label, that is, the point the person really looks at in this moment. In practice this is only roughly true. Differences between label and real gaze are called *label noise*. In short calibrations it is especially harmful, because every wrong frame is a large part of the training data.

2.9.1. Data Quality in Eye Tracking

In eye-tracking research, data quality is usually described by three measures. Niehorster et al. [40] give a recent overview of how to measure them and provide tools for it. For N

measurements in which the person looks at a known target:

- **Accuracy**: the *systematic* deviation between measured and true gaze point, usually as mean angular error

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^{N} \theta_i,$$

  where θ_i is the angle between measured and true gaze.

- **Precision**: the *spread* of repeated measurements, independent of the true target. A common measure is the RMS value between consecutive measurements

$$\text{RMS-S2S} = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N-1} \theta_{i,i+1}^2},$$

  with $\theta_{i,i+1}$ as the angle between measurement i and $i+1$, or the standard deviation of the measurements around their mean.

- **Data loss**: the share of measurements without a valid gaze value, $\text{DL} = N_{\text{invalid}}/N \cdot 100\,\%$.

The difference: an eye tracker can be precise but inaccurate (the measurements lie close together but next to the target), or accurate but imprecise (the mean is right, but the measurements spread a lot). The diagonal error of this thesis is an RMSE and therefore contains both: the systematic deviation and the spread. For learning methods, a fourth measure is added: the quality of the *training labels*. Wrong labels distort not only the evaluation but also the learned model.

2.9.2. Reaction Time and Saccades

When a calibration target appears at a new place, the gaze does not follow at once. There is a short reaction time before the *saccade* (a very fast eye movement to a new target) starts. Valliappan et al. [7] measure on smartphones a mean of 210 ms (median 167 ms). This fits earlier studies with 200 ms to 250 ms. Then follow the saccade itself and possibly small corrections until the eye rests on the target.

Frames from this *transition period* have the label of the *new* position. The gaze, however, is still on the *old* position or somewhere in between. At 30 fps, a reaction time of about 200 ms alone affects about six frames per jump. The more often the target jumps, the more frames have a wrong label.

2.9.3. Filtering During Recording

A simple counter-measure: discard the frames directly after a target change. Valliappan et al. [7], for example, always discard the first 800 ms after each new target. A fixed time window is easy to implement. But it is based on an assumption about the length of the transition, and its effect is usually not studied on its own.

This thesis takes a similar but more precise path. The recording program stores the current *phase* of the target for every frame: `WAIT` (a short waiting time of 0.5 s directly after a jump) or `MOVE` (the target then moves for 4.5 s). This makes it possible to identify the `WAIT` frames exactly and reproducibly and to remove them before training. In the recordings, this affects about 10 % of the frames. Unlike a fixed time window, this logging allows a direct comparison: models with and without cleaned data under otherwise equal conditions.

2.9.4. Filtering After Recording and Robust Learning

Other methods detect wrong labels only *after* recording, with statistical or model-based criteria.

DMRF. Jiang et al. [41] propose *Dynamic Multi-model Regression Filtering* (DMRF), a general method against wrong labels in regression tasks. It solves a *chicken-and-egg problem* (two things depend on each other, so it is unclear where to start): wrong labels distort the models used to estimate the noise, and distorted models estimate the noise wrongly. DMRF has three parts:

1. a *warm-up filtering* with a distribution function of the estimated errors,
2. a *dynamic model selection* based on a correlation criterion,
3. a *data recall* step that brings examples with small loss back to the clean data.

Besides 15 benchmarks from UCI and KEEL and an age estimation task, DMRF is built into the gaze method L2CS-Net. On a pre-processed MPIIGaze dataset with 45 000 images (leave-one-person-out), the mean error drops from 3.92° to 3.85°, while 74.3 % of the data are kept. The code is public.

SeeTN. Peng et al. [42] are the first to study systematically how wrong labels affect domain generalization in gaze estimation. In a pilot experiment they corrupt 10, 20 and 30 % of the Gaze360 labels with strong Gaussian noise. The error of a ResNet-18 baseline on MPIIGaze then rises from 7.94° to 12.17° (20 %) and 14.38° (30 %). Their method *See-Through-Noise* builds a semantic feature space. It detects wrong examples when feature and label do not fit together, and it transfers gaze information from clean to noisy examples.

Filter before or after? Methods such as DMRF and SeeTN detect wrong labels from the data themselves. They therefore also find unexpected sources of error. But they need enough examples for a reliable estimate, compute more, and have so far only been tested on large datasets, not on short calibrations of one person. Phase logging, in contrast, only covers one *known* source of error that comes from the recording procedure – but it does so exactly and without an extra model. The two approaches complement each other (Table 2.6).

Table 2.6.: Comparison of filtering after recording (DMRF) and phase logging.

Criterion	**DMRF [41]**	**This thesis**
Strategy	After: estimate errors and filter	Before: store the phase during recording
Detection	Model predictions, error patterns, dynamic model selection	`WAIT`/`MOVE` label per frame
Errors covered	All statistically unusual examples	Only the transition after a jump
Data kept	74.3 % (MPIIGaze)	Approx. 90 %
Application	Many persons, leave-one-person-out	Webcam calibration of one person
Extra effort	Several regression models, iterative training	No extra model
Result	$3.92° \rightarrow 3.85°$	Diagonal error in %, not directly comparable

2.10. Comparison and Critical Discussion

Table 2.7 summarizes the most important works of this chapter. The columns are: goal, features used, type of personalization and handling of label quality.

Critical discussion. The comparison leads to four observations:

1. **Features and personalization are studied separately.** Works on large pretrained models (UniGaze, CLIP-Gaze, GazeCLIP) aim at generalization across datasets and use full fine-tuning with a lot of data. Works on personalization (FAZE, SPAZE, Chen & Shi), in contrast, use CNNs that were trained specially for gaze. How well general pretrained models can be personalized with little calibration data is still open.

2. **Few parameters are often enough.** Offset calibration, few person parameters, prompt tuning and frozen models with adapters (PIGaze) all show: a large part of the personal error can be explained with a few adapted parameters. Kumar et al. [22] give a general reason for this.

Table 2.7.: Comparison of the most important works. k = number of calibration images per person. The errors come from different protocols and metrics and are not directly comparable.

Work	**Goal**	**Features**	**Personalization**	**Label quality**	**Result**
iTracker [1]	2D gaze, mobile	CNN branches similar to AlexNet	SVR, up to 13 points	–	1.34 cm (smartphone)
FAZE [13]	Few images	DenseNet DT-ED, supervised	MAML, $k \leq 256$	–	3.14° ($k = 9$)
SPAZE [28]	Few images	CNN, latent person parameters	Small optimization problem	–	2.70° ($k = 9$)
Chen & Shi [27]	Offset calibration	CNN	Offset from one gaze point	–	up to 10 % better
Valliappan [7]	Smartphone	CNN, pretrained on GazeCapture	Fine-tuning + regression, approx. 30 s	Fixed 800-ms discard	0.46 cm
CRGA [35]	Unsupervised DA	ResNet (ImageNet)	Pseudo-labels	–	up to 55.8 % better
UniGaze [15]	Generalization	MAE-ViT, 1.6 million faces	Full fine-tuning	–	4.65–5.86° (DG)
CLIP-Gaze [25]	Generalization	CLIP (frozen) + ResNet-18	–	Gaze-irrelevant factors	6.97° (DG)
GazeCLIP [17]	Supervised regression	CLIP ResNet-50 (fine-tuned)	–	–	3.5° (MPI-IFaceGaze)
TTP-GE [16]	Test-time adaptation	GazeDPTR V2	Self- + weakly supervised	–	6.00° (IVGaze)
TPGaze [33]	Test-time adaptation	ResNet-18	Prompt (¡1 % of parameters)	–	–
DMRF [41]	Filter wrong labels	L2CS-Net	–	After, model-based	3.85° (MPIIGaze)
SeeTN [42]	Robust generalization	ResNet-18/50	–	After, feature-based	–
This thesis	Webcam personalization	ResNet-18 (ImageNet), frozen	Head only; 30 s to 120 s	Before, phase logging	Diagonal error in %

3. **Wrong labels are usually handled afterwards.** DMRF and SeeTN filter or weight wrong labels in large datasets. Errors caused by the procedure of short calibrations are handled, if at all, with fixed time windows (Valliappan et al.), without studying their effect in detail.

4. **The results are hard to compare.** The works report angles in degrees, distances in centimetres or relative improvements, and they use different datasets, splits and protocols. An angular error can only be converted into a screen error if the geometry (distance, screen size) is known. The metric of this thesis, the Euclidean error relative to the screen diagonal, is therefore not directly comparable with the literature. The numbers in Table 2.7 are for orientation, not a ranking.

2.11. Research Gaps and Link to This Thesis

2.11.1. Research Gaps

The state of the art shows four research gaps:

G1: Data-efficient personalization of pretrained models. Large pretrained models (ImageNet CNNs, DINOv2, CLIP, MAE or UniGaze) have so far been studied for gaze estimation mainly with full fine-tuning on large datasets. Studies on how well they can be adapted to one person with only a few minutes of webcam calibration are missing.

G2: Lightweight adaptation for personalization. Several works show that a few adapted parameters capture most of the personal error. Still, head-only training, adapters and LoRA have hardly been compared systematically for personalization in gaze estimation.

G3: Label quality of short calibrations. Reaction time and target changes produce calibration frames with a wrong label. Existing work mostly handles wrong labels afterwards in large datasets or discards fixed time windows. How an exact, protocol-based removal of such frames affects personalization has not been studied yet.

G4: Realistic evaluation under webcam conditions. Many results come from benchmarks with controlled conditions or from random splits on image level, which can overestimate generalization. How camera, glasses, background and calibration length affect personalization is hardly documented.

2.11.2. What This Thesis Contributes

This thesis addresses the gaps with a simple and reproducible personalization pipeline for webcam eye tracking:

- **Frozen backbone with a small regression head (G1, G2):** A ResNet-18 pre-trained on ImageNet stays frozen. Its features are stored, and only a small regression head is trained per person. I also test whether training more layers helps. Other backbones (DINOv2, CLIP, MAE, UniGaze) and adapters or LoRA are the next step (Chapter 6).
- **Phase logging and cleaning (G3):** The recording program stores the phases `WAIT` and `MOVE`. This allows a controlled measurement of the effect of removing transition frames: models with original and with cleaned data are compared under otherwise equal conditions.
- **Systematic ablations:** Batch size, learning rate, trainable layers, activation, loss function (including an L1 loss weighted by position) and amount of data are studied with repeated runs and confidence intervals.
- **Realistic evaluation (G4):** The base models are evaluated with the subject-disjoint split `norm_subject`. Personalization is tested on 18 sessions with changing webcam (two devices), glasses, background and calibration length, with both a random and a temporal split.

Chapter 3 describes the methodology and Chapter 4 the results. Chapter 5 uses them to answer the research questions from Chapter 1.

3. Methodology

This chapter gives an overview of the procedure: data (Sections 3.1 and 3.2), pre-processing (Section 3.3), model and training (Section 3.4), evaluation (Section 3.5) and implementation (Section 3.6). The exact settings of the experiments are given in Section 4.1.

3.1. Datasets

The thesis uses two data sources:

- **GazeCapture** [1] for the base models. The full dataset has 2 445 504 frames of 1474 persons. This thesis uses a publicly available subset from Kaggle [43]. The script `main.py` downloads it and reads, for every person, the files `frames.json` (frame names) and `dotInfo.json` (position of the dot on the screen, `XPts`/`YPts`). Persons with missing files or with different numbers of frames and labels are skipped. This gives 347 085 labelled frames of 262 persons on iPhones and iPads (`labels.csv`). There are two splits, each 80/20: `norm_subject` (80 % of the persons in training, 20 % in test; no person in both) and `norm_random` (random by groups of person and target point; the same person can appear in training and test). For the experiments, 1000 training and 200 test frames are drawn at random, plus 200 validation frames of a further person.
- **Own recordings** for personalization. One person (the author) recorded 27 calibration sessions with two laptop webcams. 18 of them are evaluated (Section 3.2).

3.2. Recording the Personalization Data

A recording program shows a red point on the screen, and the person follows it with the eyes. At the same time, the webcam records images. For each image, the position of the point is stored, with 6 labels per second. The labels are in the file `norm_labels.csv` of each session; the positions are already normalized to $[0, 1]^2$. Three generations of the program are used:

- **Gen 1** (sessions 01–04): the point moves smoothly and without jumps across the whole screen.

- **Gen 2** (sessions 05–09): the point jumps every 1.4 to 2.7 seconds to a random new place. The recordings are short (about 30 s).
- **Gen 4** (sessions 19–27): the recording consists of segments of 5 s. Each segment starts with 0.5 s `WAIT` at a new start point; then the point moves for 4.5 s (`MOVE`). 90 % of the start points lie in the corners. The program stores the phase for every frame.

Gen 3 (sessions 10–18) was discarded because the model could not learn anything from it. The sessions also differ in light, background, glasses, length (90 or 120 s in Gen 4) and device. Two laptops, each with a built-in webcam, were used: *device A*, a Microsoft Surface Pro 9 (screen area 1430 × 880 pixels), for sessions 01–03 and 25–27, and *device B*, an HP laptop (1870 × 1000 pixels), for sessions 05–09 and 19–24. For session 04 the device was not logged. On both devices the frames are stored at 640 × 480 pixels. Table 4.12 shows all conditions.

3.3. Pre-Processing

- **Targets:** the gaze targets of GazeCapture are divided by the width and height of the respective screen. So all targets lie in the unit square $[0, 1]^2$, independent of the device (Section 4.1.4).
- **Images:** the whole camera image is resized to 224×224 pixels and normalized with the ImageNet means and standard deviations. There is no face or eye crop.
- **Cleaned labels:** for Gen 4, the `WAIT` frames can be removed. The script `session_design.py --write-clean` writes the file `norm_labels_clean.csv` for this.

3.4. Model and Training

The model has two parts:

1. a **ResNet-18** [2], pretrained on ImageNet (`ResNet18_Weights.DEFAULT`), as feature extractor. It gives 512 features per image.
2. a **regression head**: an MLP of four fully connected layers $512 \to 512 \to 256 \to 128 \to 2$ with ReLU, dropout $p = 0.2$ after the second layer and an output activation (default: sigmoid). It computes the position (x, y) from the features.

Training uses L1 loss (`nn.L1Loss`), AdamW (learning rate 10^{-4}, weight decay 10^{-5}), batch size 32 and early stopping (stop after three epochs without improvement, at most 500 epochs). The procedure has two stages:

1. **Base model (E0):** the head is trained on GazeCapture (`train_base.py`). Of five seeds, the run with the lowest validation error is kept.
2. **Personalization (E1–E3):** the head of the base model is trained further with the frames of one own session (`personal_exp.py`). The backbone stays frozen, including the BatchNorm statistics.

Because the backbone is frozen, its features are computed only once and stored (feature cache). After that, only the head is trained. A run therefore takes only seconds.

3.5. Evaluation

- **Error measure:** diagonal error E_{diag}, that is, the Euclidean RMSE in percent of the screen diagonal (Equation (4.1)).
- **Baselines:** the constant baseline always predicts the mean of the training targets; the random baseline guesses at random. Both are computed on the same test data as the model (Section 4.1.5).
- **Statistics:** for GazeCapture there are five repeats per setting with a 98 % confidence interval (Student's t) and the Mann-Whitney U test. For the sessions, I use the Wilcoxon signed-rank test (paired) and the Kruskal-Wallis test (comparison of groups).
- **Splitting the session data:** random 70/15/15, sequential (temporal blocks with a gap of one second) and two videos (training on one session, test on another) (Section 4.5).
- **Spatial analysis:** errors in the centre, at the edges and in the corners of the screen (Section 4.3.1).

3.6. Implementation

The code is written in Python with PyTorch and torchvision. The most important parts are:

- `main.py`: loads GazeCapture, normalizes the targets and creates the splits;
- `model.py`, `dataset.py`: model, losses, training, data loading and feature cache;

- `metrics.py`: diagonal error and baselines (one single source for all scripts);
- `exp.py`: hyperparameter study with five repeats;
- `train_base.py`, `personal_exp.py`: base models and personalization (E0–E3);
- `session_design.py`: coverage, transition frames and cleaned labels;
- `spatial_analysis.py`, `plot_targets.py`: figures;
- `auswertung.py`: creates the tables from the result files;
- notebook `notebooks/Masterarbeit.ipynb`: runs all experiments and creates tables and figures.

All experiments run without a GPU on the CPU of the HP laptop (device B): AMD Ryzen 7 3700U, about 13 GB usable memory, Ubuntu 26.04.1 LTS, Python 3.12.3, PyTorch 2.12.1 and torchvision 0.27.1 (CPU versions). The commands to repeat the experiments are given in the appendix (Appendix A.1). The code and the web demo are available on GitHub: `https://github.com/RafeiKAr/gaze-estimation`. A copy is stored in the BHT cloud: `https://cloud.bht-berlin.de/index.php/s/P4GkQ9J8zWHbdf9`.

4. Experiments and Results

This chapter describes the experiments on the GazeCapture dataset [1] and on my own recordings. First, I explain the model, the data and the evaluation (Section 4.1). Then follow the training settings (Section 4.2), the spatial distribution of the error and the loss functions (Section 4.3), and finally the personalization (Sections 4.4 to 4.6). Section 4.7 summarizes what the results mean.

All numbers were checked against the log files of the experiments. During this check, a bug in the evaluation code was found and fixed (Section 4.1.2). All values in the tables use the corrected metric. The learning curves in Figure 4.1 (bottom) and Figure 4.2, however, come from older single runs with a different data selection; the captions name the differences.

4.1. Experimental Setup

Model, training, data and evaluation are described in Chapter 3. This section only gives the extra details needed for the experiments. Table 4.1 shows the default values.

4.1.1. Data and Splits

The subset of GazeCapture and the two splits `norm_subject` and `norm_random` are described in Section 3.1. In `norm_random`, all frames of a person for the same target point are together in training or in test. For the hyperparameter study, I draw 1000 training and 200 test frames *randomly* from the split files (fixed seed 0, `--sample-seed 0`). For `norm_subject`, the training frames come from 179 persons and the test frames from 45 other persons. Older runs simply took the *first* rows of the sorted files. These were only five training persons and one test person. I do not use these runs any more.

4.1.2. Error Measure

All gaze targets lie in the unit square $[0, 1]^2$ (Section 4.1.4). The error is the square root of the mean squared *Euclidean* distance between prediction $\hat{\boldsymbol{p}}_i$ and target $\boldsymbol{p}_i$. I give it in

percent of the screen diagonal $\sqrt{2}$:

$$E_{\text{diag}} \;=\; 100 \cdot \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\hat{\boldsymbol{p}}_i - \boldsymbol{p}_i\|_2^2}}{\sqrt{2}} \quad [\%]. \tag{4.1}$$

0 % means a perfect prediction, 100 % an error as large as the whole screen diagonal.

4.1.3. Procedure

The hyperparameter study is started with the command

```
python exp.py --experiment all --sample-seed 0 --val-size 200
```

I always change only one value (for example the batch size). All other values stay at the default setting (Table 4.1). Each setting is trained five times. Early stopping uses its own validation set of 200 frames of a further person. I report the *test* error at the epoch with the lowest validation error. So the test data are never used for model selection. The tables show the mean test error, the two-sided 98 % confidence interval (CI) of the mean (Student's t, four degrees of freedom) and the mean run time.

What the "run time" measures. The run time is the real duration (wall clock) of *one whole training run*, averaged over the five runs. A run starts with the evaluation of the untrained model (epoch 0) and ends when early stopping stops. It contains all epochs and, after each epoch, the evaluation on training, validation and test data. The run time is *not* the time per epoch. It does *not* contain loading the images or the one-time computation of the backbone features before the run. Because early stopping stops at different times, the run time also shows how long a setting keeps improving.

Table 4.1.: Default setting of `exp.py`. Each variation changes exactly one of these values.

Parameter	Value	Parameter	Value
Split	`norm_subject`	Trainable layers	FC only
Train / test size	1000 / 200[a]	Output activation	Sigmoid
Batch size	32	Loss	L1 (`standard`)
Learning rate	10^{-4}	Max. epochs / patience	500 / 3
Weight decay	10^{-5}	Validation set	200 frames[b]
Repeats	5		

[a] drawn at random, seed 0. [b] from a person who is neither in training nor in test.

How to read the tables. In each hyperparameter table, the chosen option is marked with †. The column "p vs. †" gives the p-value of a two-sided Mann-Whitney U test between the five runs of this option and the five runs of the chosen option. I use a rank test because with five runs one cannot check whether the values are normally distributed, and because some options have single outlier runs (for example a ReLU run with 46.98 %). With $5 + 5$ runs the smallest possible p-value is 0.008. It means: *every* run of one option is better than *every* run of the other. A mean is printed **bold** only if the chosen option is *significantly better* than every other option in the table ($p < 0.05$ and a lower mean).

Three points are important here:

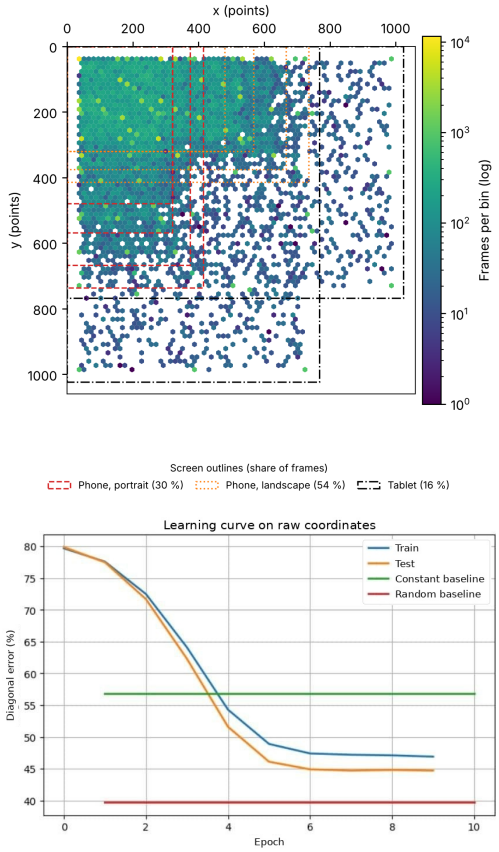
1. **Same data in all repeats.** The seed fixes the training, test and validation data. The five repeats differ only in the starting weights, the order of the data and the dropout. So the CI shows the randomness of training, not the effect of other data subsets.
2. **Small validation set.** The 200 validation frames come from only one person. Early stopping is therefore somewhat random. The test data, however, are never used for selection.
3. **Training time.** For "FC only", the head is trained on stored ResNet features (512 numbers per frame instead of the image). A run therefore takes only about one second, and differences in these times do not matter. For FC + layer 4 and the full ResNet, every image goes forward and backward through the backbone in every epoch. These times are much longer and not directly comparable with "FC only".

4.1.4. Normalization of the Targets

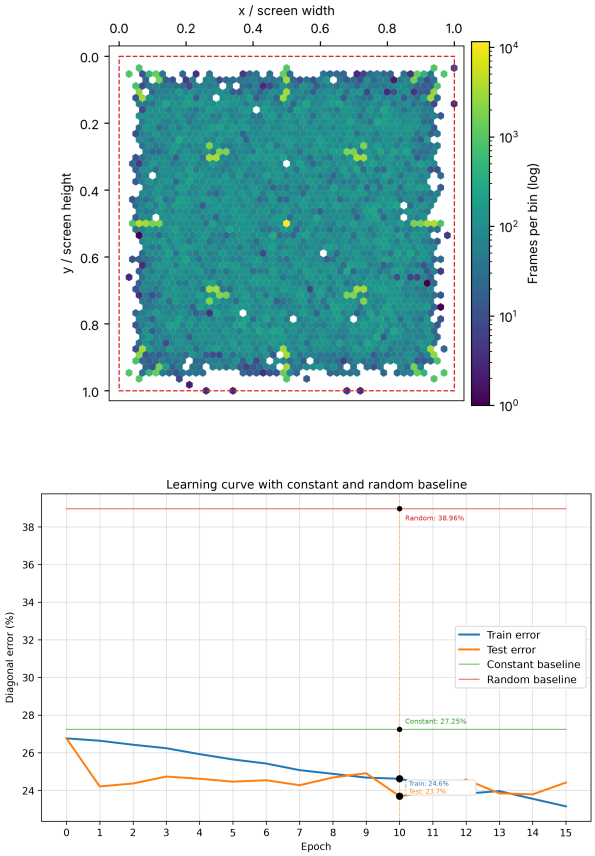
The raw gaze targets in GazeCapture are screen points of the device, measured from the top left corner. Their range therefore depends on screen size and orientation. The 347 085 frames of the 262 persons come from ten screen formats, from 320×480 to 1024×768 points, in portrait and landscape. I therefore divide the targets by the screen width W and height H of each frame (`main.py`; values outside $[0, 1]$ are clipped, which affects 0.006 % of the frames):

$$x_{\text{norm}} = \frac{x}{W}, \qquad y_{\text{norm}} = \frac{y}{H}. \tag{4.2}$$

Figure 4.1 shows the effect. Without normalization, the targets of the many smartphone frames lie mainly at the top left, and the same coordinate means a different place on every device (Figure 4.1a). The error values are then not comparable between devices (left column). After normalization, all targets lie in $[0, 1]^2$, and Equation (4.1) is well-defined (right column). All further experiments use normalized targets.



(a) Raw targets in screen points (density of all frames, logarithmic colour scale); dashed: outlines of the screen formats. Bottom: learning curve of an early notebook run on raw coordinates; these values are not percentages of a common diagonal and not comparable with the other figures and tables.



(b) Normalized targets (same display; dashed: unit square). The bright cells are calibration points that are shown at the same relative places on all devices. Bottom: learning curve of an older notebook run on `norm_random` (1000/200, older data selection, see Section 4.1.1). Epoch 0 is the untrained model. The horizontal lines are fixed baselines of the *whole* dataset (27.25 % and 38.96 %), not the baselines of the test data of this run (constant baseline there: 26.76 %, see text). The values are not directly comparable with the tables.

Figure 4.1.: Raw and normalized gaze targets compared.

Why does the curve start at the constant baseline and not at the random baseline? In Figure 4.1b, the curves start at epoch 0, that is, *before* the first training step, at 26.77 %. One could expect that an untrained model is as bad as random guessing (about 40 %). This is not true.

The random baseline guesses a *different* random point on the screen for each image. An untrained network does not do that. Its weights are small random numbers. The last linear layer therefore outputs values close to 0 for every image, and the sigmoid turns them into $\sigma(0) = 0.5$. So the untrained model predicts almost the *same* point for all images: the screen centre $(0.5, 0.5)$. This is exactly what the constant baseline does. Correct is therefore:

- epoch 0 $\approx$ constant baseline of the *same* test data ("always look at the centre");
- only when training brings the curve *below* this value has the model learned something from the images;
- the random baseline is an upper limit for "pure guessing"; a model should never be above it.

The measured values confirm this: on the 200 test images of this run, the constant baseline is 26.76 % (mean of the training targets) or 26.71 % (screen centre). This matches the start value of 26.77 %. It is important to compute the baseline on the *same* data as the curve. A baseline on the whole dataset (27.25 %) is somewhat higher, and the curve then seems to start "below the baseline" – as with the horizontal line in Figure 4.1b. The code therefore now computes the baselines on the same subset (`metrics.subset_baselines`).

4.1.5. Baselines

I use two simple reference estimators. The *constant baseline* predicts the mean of the training targets for every test image. The *random baseline* draws predictions uniformly from $[0, 1]^2$. A model is only useful if it is clearly better than the constant baseline.

Table 4.2.: Baseline errors E_{diag} in %. Top: whole dataset (`exp.py`, bootstrap with 30 samples of 2000 frames each, mean and 98 % CI). Bottom: the same random training/test subsets (seed 0) as used for model training (no CI, one fixed subset).

Data	Constant baseline	Random baseline
Whole dataset (347 085 frames)	27.28 [27.15, 27.40]	39.82 [39.60, 40.04]
`norm_subject`, 1000/200	28.46	40.36
`norm_subject`, 10 000/2000	27.62	39.96
`norm_random`, 1000/200	27.66	39.83

The lower part of Table 4.2 is the correct reference for all model results in Section 4.2, because it was computed on the same data.

4.2. Training Settings

All results in this section come from one run (`exp_results_all.json`; `exp.py`, corrected metric, random subsets with seed 0, early stopping on validation, five repeats per setting). The constant baseline of the default setting on the same data is 28.46 %.

4.2.1. Batch Size

Table 4.3.: Batch size. Mean test error over five runs with 98 % CI and mean duration of a whole training run (head on stored features, see Section 4.1.3).

Batch size	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
32†	28.39	[28.21, 28.58]	—	0.019
64	28.35	[28.29, 28.41]	0.69	0.012
128	28.38	[28.30, 28.47]	0.84	0.009

The three batch sizes give the same test error (Table 4.3): the means differ by at most 0.05 percentage points, the confidence intervals overlap, and the rank test finds no difference ($p \geq 0.69$). A run takes less than one second, because only the small head is trained on stored features. The time differences therefore do not matter.

I keep batch size 32 as the default. The experiment shows no disadvantage, and two practical reasons speak for a small batch size in personalization: (i) a personal session has only a few hundred training images, and small batches give more updates per epoch; (ii) the noise of small batches is commonly thought to have a slight regularizing effect. The data of this experiment, however, do not *prove* an advantage of 32 over 64 or 128.

4.2.2. Learning Rate

Table 4.4.: Learning rate. Mean test error over five runs with 98 % CI and mean duration of a whole training run.

Learning rate	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
10^{-3}	28.43	[28.33, 28.54]	0.15	0.018
10^{-4}†	28.37	[28.32, 28.42]	—	0.020
10^{-5}	28.33	[28.28, 28.38]	0.056	0.022
10^{-6}	28.37	[28.21, 28.52]	0.42	0.055

The learning rate also shows no significant difference (Table 4.4): all means lie between 28.33 % and 28.43 %, the intervals overlap, and no rank test is significant ($p \geq 0.056$). So

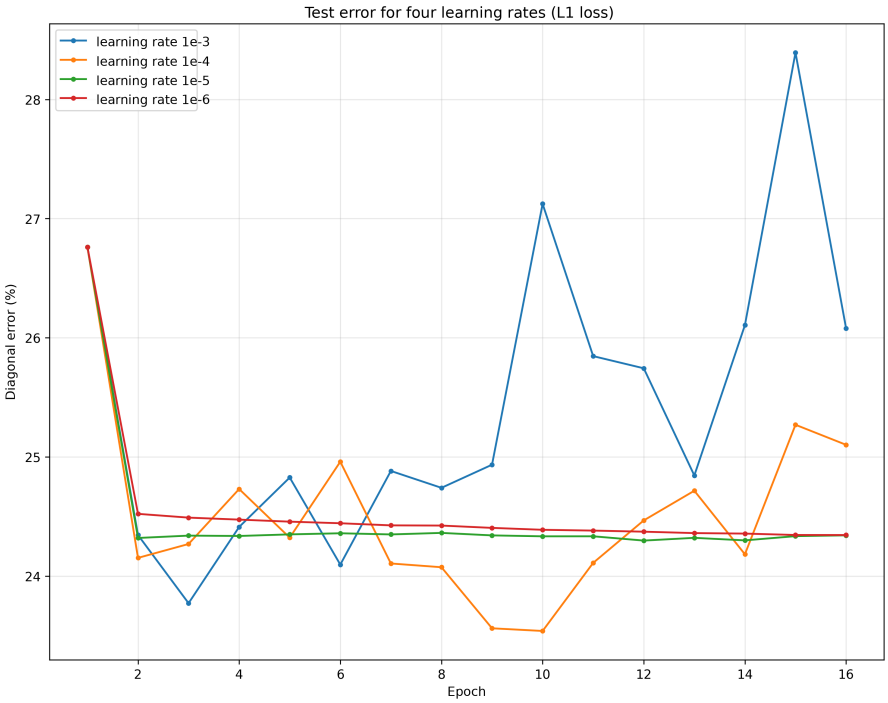


Figure 4.2.: Test error of single training runs with four learning rates from an older notebook run (`norm_random`, 1000/200, older data selection, 15 epochs without early stopping). The first point (epoch 1 on the axis) is the untrained model. The values are not directly comparable with Table 4.4.

the final error alone cannot justify the choice. The deciding factor is the first test, i.e. the *learning behaviour* in Figure 4.2:

- $\boldsymbol{\eta = 10^{-3}}$: The steps are too large. After a fast drop, the test error jumps up and down and rises again (peaks at about 27.1 % and 28.4 %). The run does not find a stable minimum.
- $\boldsymbol{\eta = 10^{-4}}$: clearly reaches the lowest test error of all curves (about 23.6 % around epoch 10, i.e. about 0.7 percentage points below 10^{-5} and 10^{-6}). The curve still moves, but much less than for 10^{-3}. Early stopping can keep this best state.
- $\boldsymbol{\eta = 10^{-5}}$ **and** $\boldsymbol{10^{-6}}$: The curves are very smooth and fall steadily, but stop at about 24.3 %. The steps are too small to get further in the available time. The result is stable, but not the best.

The run time also speaks for 10^{-4} (Table 4.4): with 0.020 min (about 1.2 s) it is among the fastest values. Only 10^{-3} is marginally faster, but unstable; 10^{-6} needs almost three times as long (0.055 min), because training with very small steps stops later. I therefore choose $\eta = 10^{-4}$: lowest error in the first test, a calm learning behaviour and a short run time. The repetition with five runs also shows that the head on stored features is robust to the learning rate: none of the other values is significantly better.

4.2.3. Trainable Layers

I compare three depths of fine-tuning (Table 4.5): only the regression head (FC only), the head plus the last residual stage (`layer4`), and the whole network.

Table 4.5.: Trainable parameters of the three variants (ResNet-18 with regression head, 11 603 650 parameters in total).

	FC only	FC + layer 4	Full ResNet
Trainable parameters	427 138	8 820 866	11 603 650
Share of all parameters	3.68 %	76.02 %	100.00 %
Backbone features can be stored	yes	no	no

Table 4.6.: Trainable layers. Mean test error over five runs with 98 % CI and mean duration of a whole training run. For "FC only", no image passes through the backbone (stored features).

Trainable layers	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
FC only†	28.36	[28.23, 28.49]	—	0.015
FC + layer 4	29.07	[26.58, 31.55]	0.22	9.160
Full ResNet	29.39	[26.99, 31.78]	0.31	15.366

"FC only" has the lowest mean error and by far the narrowest interval (Table 4.6). More trainable layers do not help. They even make the result *less reliable*: one run with FC + layer 4 (31.72 %) and two runs with the full ResNet (30.70 % and 31.18 %) end clearly above the constant baseline (28.46 %). All runs with "FC only" stay between 28.28 % and 28.47 %. With only five runs, the differences are not significant ($p = 0.22$ and 0.31), so the value for "FC only" is not bold. Training time grows strongly with the number of parameters: 9.2 min per run for FC + layer 4 and 15.4 min for the whole network.

I choose "FC only" for three reasons:

1. **Not worse, but more stable:** more layers do not lower the error here; they only increase the spread between runs.
2. **Robust with little data:** a personal session has only a few hundred images. Training millions of backbone parameters with so few examples risks overfitting and distorts the pretrained features. Kumar et al. [22] show that full fine-tuning can damage pretrained features and generalize worse than head-only training.
3. **Effort:** only 3.68 % of the parameters are trained, and the backbone features are computed only once. This makes personalization fast and memory-friendly.

4.2.4. Output Activation

The output activation comes after the last linear layer, which outputs (x, y). Sigmoid maps to $(0, 1)$, which is exactly the range of the normalized coordinates. The Gaussian activation $g(z) = \exp(-z^2/2\sigma^2)$ with $\sigma = 1$ maps to $(0, 1]$, but it is not monotonic. ReLU only removes negative values, and "None" leaves the output unchanged.

Table 4.7.: Output activation. Mean test error over five runs with 98 % CI and mean duration of a whole training run.

Activation	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
Sigmoid†	**28.36**	[28.29, 28.43]	—	0.022
Gaussian	31.29	[28.44, 34.14]	0.008	0.019
ReLU	32.90	[19.69, 46.10]	0.008	0.028
None	29.39	[28.42, 30.36]	0.008	0.032

Sigmoid is the only clear winner of the hyperparameter study (Table 4.7): lowest error, narrowest interval, and *every* sigmoid run is better than *every* run of the three other activations ($p = 0.008$ in all three comparisons). The value is therefore bold. The other activations are also less stable: one ReLU run ends at 46.98 %, which is worse than random guessing. A possible reason: ReLU can output zero for all inputs ("dead" output), and the gradient does not recover from this. Without a bounding activation (None, ReLU), the

network can also predict points outside the screen. I choose sigmoid because it keeps the output on the screen and gives the lowest and most stable error.

4.2.5. Amount of Data

Table 4.8.: Amount of data (training/test). Mean test error over five runs with 98 % CI, mean duration of a whole training run and constant baseline of the respective test data.

Training / test	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)	Const. baseline (%)
10 000 / 2000	27.64	[27.55, 27.73]	0.008	0.167	27.62
1000 / 200†	28.39	[28.33, 28.44]	—	0.018	28.46

The larger amount of data has a lower mean error (Table 4.8). But the two variants are evaluated on *different* test data, and each mean is practically equal to the constant baseline of its own test data (27.64 % vs. 27.62 % and 28.39 % vs. 28.46 %). The difference of 0.75 percentage points therefore comes from the different test data and not from more training data – even though the rank test is significant ($p = 0.008$). I keep the small variant (1000/200) because it is about ten times faster and has about as much data as a personalization session.

4.2.6. Type of Split

Table 4.9.: Type of split. Mean test error over five runs with 98 % CI and constant baseline of the respective test data.

Split	Mean test error (%)	98 % CI (%)	p vs. †	Constant baseline (%)
`norm_random`	27.72	[27.64, 27.80]	0.008	27.66
`norm_subject`†	28.39	[28.26, 28.53]	—	28.46

The random split gives the lower error (Table 4.9). Here, too, most of the difference comes from the test data: the constant baseline is already 0.80 percentage points lower on `norm_random`, and both models lie at most 0.07 percentage points away from their baseline. In addition, the random split allows the same persons in training and test. This makes the task easier and the result too optimistic. I therefore use `norm_subject` as the default. Only this split measures the performance on unknown persons, that is, the situation before personalization.

4.3. Spatial Error Analysis and Loss Functions

The mean error from Section 4.2 does not show *where* on the screen the errors occur. This section first studies the spatial distribution of the error (Sections 4.3.1 and 4.3.2). The result led to the idea of weighting the loss by position (Section 4.3.3). Section 4.3.4 evaluates this idea.

4.3.1. Screen Partition

I describe the normalized screen $[0,1]^2$ with a 3×3 grid of three regions:

$$\begin{aligned} \text{Centre:} \quad & (0.50, 0.50), \\ \text{Edges:} \quad & (0.10, 0.50),\ (0.50, 0.10),\ (0.90, 0.50),\ (0.50, 0.90), \\ \text{Corners:} \quad & (0.10, 0.10),\ (0.10, 0.90),\ (0.90, 0.10),\ (0.90, 0.90). \end{aligned}$$

A test example with target $\boldsymbol{p}$ belongs to grid point $\boldsymbol{g}$ if $\|\boldsymbol{p} - \boldsymbol{g}\|_2 \leq r$ with $r = 0.15$. The error of a grid point is the mean Euclidean error $\|\hat{\boldsymbol{p}} - \boldsymbol{p}\|_2$ of its examples (in normalized units, that is, as a share of the screen side; the diagonal is $\sqrt{2} \approx 1.41$). The error of a region is the mean over all examples of its grid points. These definitions apply to all figures and tables in this section.

4.3.2. Error Heat Maps

Figure 4.3 uses the 200 test predictions of the default model (`norm_subject`, L1 loss; file `results/heatmap/base_loss1.csv`). For a position $\boldsymbol{c}$ (centre, edge or corner), I compute the mean error $\bar{e}(\boldsymbol{c})$ of all examples within radius r and draw it as a Gaussian spot:

$$H(x, y) = \bar{e}(\boldsymbol{c}) \exp\left(-\frac{\|(x, y) - \boldsymbol{c}\|_2^2}{2\sigma^2} \right), \qquad \sigma = 0.20. \tag{4.3}$$

So the colour shows the *size* of the local error, and the position of the bright spot shows *where* it was measured. All three panels use the same colour scale (from 0 to the largest value), so the colours can be compared directly.

The error grows from the inside to the outside: it is 0.086 in the centre ($n = 18$), 0.432 at the edges ($n = 62$) and 0.586 in the corners ($n = 42$). The edge errors are about five times and the corner errors about seven times as large as in the centre.

The fourth panel explains the reason. All predictions lie in a very small area around $(0.5, 0.5)$. Their standard deviation is only about 0.004 in both coordinates, while that of the targets is about 0.28. So the model predicts almost the same point for every image. Its

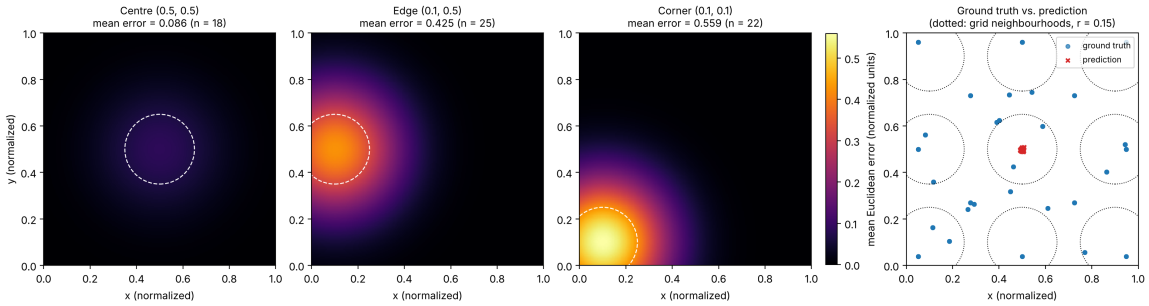


Figure 4.3.: Spatial error of the default model (L1 loss, `norm_subject`, 200 test examples). Panels 1–3: error heat maps ($r = 0.15$, $\sigma = 0.20$, common colour scale) at the centre $(0.5, 0.5)$, the left edge $(0.1, 0.5)$ and the bottom left corner $(0.1, 0.1)$. The error grows from the centre (0.086) over the edge (0.425) to the corner (0.559). Panel 4: targets (blue) and predictions (red); all predictions lie close to the screen centre.

error is simply the distance of the target from the screen centre. An estimator that always outputs the centre gives practically the same values (centre 0.085, edges 0.432, corners 0.587; in total $E_{\text{diag}} = 28.45\,\%$ compared with $28.47\,\%$ for the model).

4.3.3. Loss Functions

Because of the large errors at the edges and corners, the idea came up to give these areas more weight in the loss. Let $\hat{\boldsymbol{p}}_i = (\hat{x}_i, \hat{y}_i)$ be the prediction and $\boldsymbol{p}_i = (x_i, y_i)$ the target of example i in a batch of size N.

L1 loss (`nn.L1Loss`, "standard").

$$\mathcal{L}_1 = \frac{1}{2N} \sum_{i=1}^{N} \big(|\hat{x}_i - x_i| + |\hat{y}_i - y_i| \big). \tag{4.4}$$

The gradient always has the same size, no matter how large the error is. The loss is therefore robust against outliers (for example wrong labels during target changes), but it treats all screen positions the same.

Smooth L1 loss (`nn.SmoothL1Loss`, $\beta = 1$). For a residual u:

$$\ell_\beta(u) = \begin{cases} \dfrac{u^2}{2\beta}, & |u| < \beta, \\ |u| - \dfrac{\beta}{2}, & \text{otherwise.} \end{cases} \tag{4.5}$$

It is quadratic for small and linear for large residuals. Because all residuals here are smaller than $\beta = 1$, it behaves like a scaled squared error: for small errors the gradient becomes small.

Sensitive L1 loss (weighted by position, `sensitive`). Each example is weighted with its L_1 distance to the screen centre:

$$d_i = |x_i - 0.5| + |y_i - 0.5| \in [0, 1], \qquad w_i = 1 + d_i \in [1, 2], \tag{4.6}$$

$$\mathcal{L}_{\text{sens}} = \frac{1}{N} \sum_{i=1}^{N} w_i \cdot \frac{|\hat{x}_i - x_i| + |\hat{y}_i - y_i|}{2}. \tag{4.7}$$

An example in a corner counts twice as much as one in the centre, an example at an edge about 1.4 times as much. The gradient for outer targets becomes larger, and this should pull the predictions away from the centre. (This formula matches the implementation in `model.py`.)

The "improved sensitive loss" in Table 4.11 was another variant from a notebook for trying things out. Its exact formula is no longer in the current code. I list it only for completeness.

4.3.4. Results of the Loss Comparison

Table 4.10 shows the global comparison with five runs together with the errors in the screen regions from Section 4.3.1.

Table 4.10.: Comparison of L1 and sensitive L1 loss. Left: mean test error E_{diag} over five runs (`exp.py`, 98 % CI) and mean run time. Right: mean Euclidean error per screen region (normalized units; one run each, 200 test examples, $r = 0.15$; n = number of examples). Last row: estimator that always outputs the centre, on the same 200 test examples.

	Global (5 runs)			Spatial error (1 run)		
Loss	E_{diag} (%)	98 % CI (%)	Run time (min)	Centre ($n = 18$)	Edges ($n = 62$)	Corners ($n = 42$)
L1 (standard)[†]	**28.32**	[28.23, 28.42]	0.011	0.086	0.432	0.586
Sensitive L1	28.48	[28.19, 28.78]	0.021	0.090	0.431	0.585
Constant centre	28.45[a]	—	—	0.085	0.432	0.587

[a] On the same 200 test examples as the spatial analysis.

Table 4.11.: Additional experiment: spatial error of four loss functions after 15 epochs on training data with 52 corner examples (two runs, A and B, values as logged). The numbers come from a notebook for trying things out. The computation of the region values is not in the current code; they are therefore not directly comparable with Table 4.10.

	Overall (%)		Centre (%)		Edges (%)		Corners (%)	
Loss	A	B	A	B	A	B	A	B
L1 (`nn.L1Loss`)	24.54	24.63	17.01	16.74	28.28	28.98	34.93	35.07
Smooth L1 (`nn.SmoothL1Loss`)	23.94	24.91	15.40	16.41	29.78	30.59	34.13	35.17
Sensitive L1	25.57	25.55	19.11	18.61	29.87	29.93	33.41	34.20
Improved sensitive L1	25.53	27.75	19.20	21.86	29.17	35.03	33.77	31.58

Observations.

- Globally, L1 is slightly but significantly better than the sensitive loss: 28.32 % compared with 28.48 % (Mann-Whitney $p = 0.032$, Table 4.10); the L1 value is therefore bold. The sensitive loss also varies more between runs. Both values are close to the constant baseline (28.46 %).
- Spatially, the weighting changes the region errors by less than 0.005 normalized units. Edge and corner errors stay about five to seven times as large as in the centre.
- Both models have the same spatial pattern as an estimator that always outputs the centre. The weighting does increase the gradient for outer examples, but this is not enough for the model to leave the centre.
- In the additional experiment (Table 4.11), the sensitive variants lower the corner error a little (sensitive L1 by 0.9–1.5, the improved variant by 1.2–3.5 percentage points). But they raise the centre error by 1.9–5.1 percentage points and the overall error in all runs. The two runs differ a lot; these trends are therefore not reliable.

Conclusion. The spatial analysis shows *what* the problem is: the errors lie mainly at the edges and in the corners. The weighted loss was meant to reduce these errors. It did not improve the corners and was globally even slightly but significantly worse than L1 ($p = 0.032$). In the additional experiment it raised the error in the centre. I therefore keep the normal L1 loss: it has the lower error, is simpler, has no extra parameter, and its constant gradient makes it robust against wrong labels in the own recordings. The analysis also shows *why* the weighting could not help: the large outer errors are not caused by a wrong weighting, but by the model hardly leaving the screen centre (Section 4.7).

4.4. Personalization Sessions

4.4.1. Setup

In each personal session, the head of a GazeCapture base model is trained further on the frames of one own recording (FC only, sigmoid, L1, $\eta = 10^{-4}$, weight decay 10^{-5}, batch size 32, AdamW). Unless stated otherwise, each session is split randomly into 70 % training, 15 % validation and 15 % test (seed 42). Early stopping (patience 3) uses the validation error. I report the test error at the epoch with the lowest validation error. If the best epoch is 0, personalization did not improve the validation error. The reported test error is then that of the unchanged base model.

All results from here on come from the script `personal_exp.py` (notebook `notebooks/Masterarbeit.ipyn` and use the corrected metric (Equation (4.1)). All random seeds are fixed. The backbone, including its BatchNorm statistics, is completely frozen.

Feature cache. Because the backbone is frozen, it gives the same 512 features for an image in every epoch. I therefore compute these features only once per session and base model (without augmentation) and store them. After that, only the head is trained. This does not change the result, but a run takes only seconds on a CPU. The only difference to older notebook runs: there is no image augmentation during personalization.

Base models (E0). The two base models are trained with the setting from Section 4.2 (FC only, sigmoid, L1, $\eta = 10^{-4}$, batch size 32, 1000/200 random frames, seed 0, early stopping on validation) on `norm_subject` and `norm_random`. Of five seeds, I keep the run with the lowest *validation* error (`train_base.py`, `models/base/base_<split>.pt`; a JSON file documents the setting and all seeds). For `norm_subject` this is seed 0 (validation 27.37 %, test 28.70 %; constant baseline of the same test data 28.46 %), for `norm_random` seed 3 (validation 26.74 %, test 27.81 %; constant baseline 27.66 %). The five seeds differ by less than 0.4 percentage points, so the choice of seed is not critical. As described in Section 4.7, both base models are at the level of the constant baseline.

Of 27 recorded sessions, I use 18: Gen 1 (01–04), Gen 2 (05–09) and Gen 4 (19–27). I discarded Gen 3 (10–18) because the model could not learn anything from it. Table 4.12 shows the recording conditions.

4.4.2. Results of All Sessions (E1)

Table 4.13 shows every session with both base models.

Table 4.12.: Recording conditions of the 18 sessions. Gen 1: target moves smoothly, without jumps; Gen 2: target with random jumps; Gen 4: 5-s segments (0.5 s WAIT at a new start point, 4.5 s MOVE), 90 % of the start points in the corners. The length was only changed in Gen 4 (Gen 1/2: not logged). Device A: sessions 01–03 and 25–27; device B: 05–09 and 19–24; session 04: not logged.

Sessions	Generation	Glasses	Light / background
01, 03, 04	Gen 1	with	01, 04: bright; 03: dark
02	Gen 1	without	bright
05, 07, 08	Gen 2	with	05: bright; 07: dark; 08: very dark
06, 09	Gen 2	without	06: bright; 09: very dark
Gen 4	(length / device)		
19, 20	Gen 4	with	calm; 120 s / 90 s; B
21, 22	Gen 4	without	calm; 120 s / 90 s; B
23 / 24	Gen 4	with / without	busy; 120 s; B
25 / 26	Gen 4	with / without	busy; 120 s; A
27	Gen 4	with	busy; 90 s; A

Table 4.13.: Personalization of the 18 sessions with two base models (original labels, random 70/15/15 split, seed 42). Base: test error of the unchanged base model (epoch 0); Val.: best validation error; Test: test error at the best validation epoch; Ep.: best epoch; t: duration of one run in seconds (Section 4.4.3); Const.: constant baseline (mean of the training targets, on the same test frames). Errors in % (Equation (4.1)). Group rows: mean and 98 % CI over the sessions of the group.

		Base model `norm_subject`					Base model `norm_random`					
Session	Gen.	Base	Val.	Test	Ep.	t (s)	Base	Val.	Test	Ep.	t (s)	Const.
01	1	29.19	14.82	15.83	22	2.3	29.49	11.70	12.97	47	4.2	29.45
02	1	28.48	10.08	10.48	30	2.2	28.42	10.04	10.43	44	3.2	28.49
03	1	29.61	15.26	17.20	33	2.5	30.13	15.41	17.35	36	2.6	29.76
04	1	29.98	16.88	17.64	17	2.1	29.38	15.23	16.16	31	2.7	29.17
05	2	32.92	25.43	17.64	26	1.1	32.49	31.93	32.77	2	0.2	32.72
06	2	29.29	11.31	16.67	54	1.8	28.89	12.09	16.55	55	2.0	27.02
07	2	32.18	24.84	30.99	16	0.7	32.43	25.02	31.62	18	0.6	32.28
08	2	25.82	23.65	23.30	24	0.8	26.32	23.38	22.31	23	0.6	27.00
09	2	26.83	23.55	28.64	4	0.2	26.21	15.19	16.87	47	1.9	28.17
19	4	30.42	13.38	17.76	33	2.8	30.04	12.13	16.85	47	4.4	30.19
20	4	33.14	16.06	14.64	23	1.9	33.01	14.58	12.93	35	2.7	33.49
21	4	30.18	15.03	12.51	39	4.6	30.46	15.80	13.20	23	2.6	29.97
22	4	31.17	13.54	13.02	22	2.2	30.87	12.53	12.32	37	3.1	30.32
23	4	33.15	23.48	23.65	10	1.4	32.59	24.02	24.05	10	2.0	32.57
24	4	33.09	28.77	26.97	7	0.9	32.84	29.53	27.42	6	1.0	33.01
25	4	32.19	26.98	24.61	19	2.4	33.00	26.74	24.13	26	2.8	31.54
26	4	30.60	21.93	18.61	20	2.3	30.39	21.40	17.97	37	4.5	29.48
27	4	31.98	26.05	26.06	19	1.5	32.29	26.27	26.68	19	1.7	31.19
Gen 1 (4)		29.32	14.26	15.29	25.5	2.3	29.35	13.10	14.23	39.5	3.2	29.22
		[27.86, 30.77]	[7.62, 20.89]	[7.80, 22.77]		[1.8, 2.7]	[27.75, 30.96]	[7.07, 19.12]	[7.10, 21.35]		[1.5, 4.8]	[27.99, 30.45]
Gen 2 (5)		29.41	21.76	23.45	24.8	0.9	29.27	21.52	24.02	29.0	1.0	29.44
		[24.14, 34.68]	[11.88, 31.63]	[12.74, 34.16]		[0.0, 1.9][a]	[24.06, 34.47]	[8.20, 34.85]	[10.93, 37.12]		[0.0, 2.5][a]	[24.68, 34.20]
Gen 4 (9)		31.77	20.58	19.76	21.3	2.2	31.72	20.33	19.51	26.7	2.8	31.31
		[30.60, 32.94]	[14.66, 26.50]	[14.26, 25.26]		[1.2, 3.3]	[30.51, 32.93]	[13.88, 26.79]	[13.59, 25.42]		[1.7, 3.9]	[29.91, 32.70]
All (18)		**30.57**	**19.50**	**19.79**	**23.2**	**1.9**	**30.51**	**19.05**	**19.59**	**30.2**	**2.4**	**30.32**
		[29.27, 31.87]	[15.87, 23.13]	[16.18, 23.40]		[1.3, 2.5]	[29.21, 31.82]	[14.86, 23.25]	[15.47, 23.71]		[1.6, 3.2]	[29.12, 31.53]

[a] The lower limit of the t interval is below zero (few sessions, large spread) and was cut at 0.

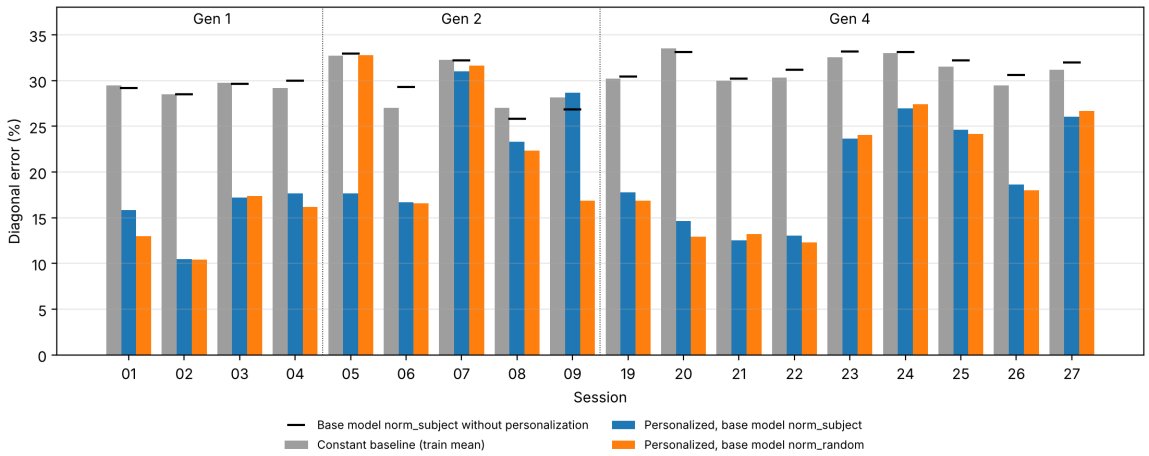


Figure 4.4.: Test error of the 18 sessions after personalization with both base models, compared with the constant baseline and the unchanged base model (`norm_subject`) on the same test frames (data from Table 4.13).

Personalization works. Over all 18 sessions, the test error drops from 30.32 % (constant baseline) to 19.79 % with the base model `norm_subject` and to 19.59 % with the base model `norm_random`. This is a reduction of 35 % and 36 % (Wilcoxon signed-rank test $p = 1.5 \cdot 10^{-5}$ for both; paired t test $p < 10^{-6}$). The unchanged base models are not better than the constant baseline (30.57 % and 30.51 %; Wilcoxon $p = 0.25$ and $p = 0.35$). So the whole gain comes from the few minutes of personal data. The best session, S02 (Gen 1, bright room, no glasses), reaches 10.5 %. The new runs confirm the older notebook runs (mean 19.85 % over the same sessions with `norm_subject`).

Groups. Compared with the constant baseline, the error drops by 48 % / 51 % in Gen 1, by 20 % / 18 % in Gen 2 and by 37 % / 38 % in Gen 4 (`norm_subject` / `norm_random`). In Gen 4, the improvement is significant (Wilcoxon $p = 0.004$). For Gen 1 ($n = 4$) and Gen 2 ($n = 5$), the smallest possible p-value of this test is 0.125 and 0.0625. These groups are therefore too small for a significant result, even though the effect in Gen 1 is large. The differences between the generations are not significant (Kruskal-Wallis $p = 0.13$ and $p = 0.12$). They must not be read as an effect of the recording procedure alone (Section 4.6).

When personalization fails. In two runs, the personalized model is not better than the constant baseline: S09 with `norm_subject` (best epoch 4, 28.64 % compared with 28.17 %) and S05 with `norm_random` (best epoch 2, 32.77 % compared with 32.72 %). In both cases the validation error stopped improving very early. With the other base model, the same sessions reach 16.87 % (S09) and 17.64 % (S05). So the failure is not caused by the session but by a single run: with only 20 to 30 validation frames (Gen 2), early stopping can end training after a few epochs. S07 (dark room) improves only a little with both base models

(31.0 % / 31.6 % compared with 32.3 %). Good light therefore remains a requirement for webcam personalization.

Base model. Both base models give the same result (19.79 % compared with 19.59 %; Wilcoxon $p = 0.20$). This fits the main result: the base model mainly provides a starting point, and the personal data decide the result. In the following experiments I use the base model `norm_subject`, because it was trained without any information about the test persons.

Validation and test error. On average, the validation error (19.50 %) is close to the test error (19.79 %). The largest differences (5 to 8 percentage points, S05 to S07, S09) occur in the short Gen 2 recordings. There, validation and test have only about 25 frames each.

4.4.3. Training Time of a Session

The time t in Table 4.13 is the real duration of *one whole personalization run* on the stored features: evaluation of the unchanged model (epoch 0), all epochs until early stopping and, after each epoch, the evaluation on training, validation and test data. It is neither a time per epoch nor a mean over several runs. It does not contain the one-time computation of the backbone features of the session. On a CPU, a run takes 1.9 s on average (`norm_subject`) and 2.4 s (`norm_random`). The duration grows with the number of frames and epochs (short Gen 2 recordings: 0.9 s). Older notebook runs, which sent every image through the backbone in every epoch, needed 1 to 26 minutes per session. The "run time" in the hyperparameter tables (Section 4.1.3), in contrast, is a mean over five runs of one GazeCapture setting.

4.4.4. Label Cleaning (Gen 4, E1b)

In Gen 4, every segment starts with a WAIT phase of 0.5 s. The target appears at a new place, and the user first has to find it. These frames (10.0 to 10.5 % of the labels) can have a wrong gaze label. Because they are logged exactly, one can write cleaned label files without the WAIT frames (`session_design.py --write-clean`).

A simple comparison would be unfair, however: the cleaned model is tested without WAIT frames, while the original model is tested on all frames, including the hard-to-predict WAIT frames. I therefore also evaluate the original model only on the MOVE frames of its test data (same model, WAIT frames removed from the test; Table 4.14).

Table 4.14.: Label cleaning in Gen 4 (base model `norm_subject`, random split, seed 42). Original: trained with all labels; test error on all test frames, on MOVE only and on WAIT only (n_W = number of WAIT test frames). Cleaned: trained and tested without WAIT frames. Δ = cleaned − original (MOVE).

	Original labels				Cleaned labels		
Session	All	MOVE	WAIT	n_W	Test	Const.	Δ
19	17.76	13.28	41.12	8	13.06	29.07	−0.22
20	14.64	12.24	28.38	8	12.77	30.71	+0.53
21	12.51	9.77	27.59	10	9.79	29.23	+0.01
22	13.02	10.21	27.64	8	9.96	31.71	−0.25
23	23.65	20.95	41.84	10	17.18	30.92	−3.77
24	26.97	24.96	41.93	10	23.99	31.64	−0.97
25	24.61	21.63	44.36	10	17.94	30.49	−3.70
26	18.61	14.34	41.72	10	14.58	30.20	+0.24
27	26.06	23.10	44.80	8	23.69	30.59	+0.60
Mean	19.76	16.72	37.71	9.1	15.88	30.51	−0.84

Result. The WAIT frames are predicted very badly (37.7 %, worse than the constant baseline). The gaze is not yet on the target, so their labels are wrong. Removing them only from the *test* data lowers the error from 19.76 % to 16.72 %. Removing them also from the *training* data lowers it further to 15.88 %. This gain of 0.84 percentage points is not significant (Wilcoxon $p = 0.43$). It comes from two sessions with a busy background (S23, S25, about −3.7 points each). In the calm sessions, the difference is smaller than 0.6 points. Conclusion: the WAIT frames should at least be removed from the evaluation, because they measure the error of the labels and not the error of the model. Removing them also from training costs nothing and helps in difficult sessions; the effect, however, is small.

Variation between runs. Two older notebook runs with the same setting on the same data (Gen 4, seeds not fixed) differed by 1.9 on average and by at most 5.5 percentage points. The new runs with three fixed seeds (Table 4.15) show a standard deviation of up to 3.1 percentage points between seeds. Differences between *single* runs of this size can therefore not be interpreted. This also explains the cases S05 and S09 above.

4.4.5. Recording Conditions (Gen 4)

In Gen 4, one can compare sessions that differ in only one factor. Each value is the difference between two single sessions (test error from Table 4.13, `norm_subject` / `norm_random`, in percentage points; positive means: the first condition is worse):

- **Glasses** (with − without): +5.2/ + 3.6 (S19–S21), +1.6/ + 0.6 (S20–S22), −3.3/ − 3.4 (S23–S24), +6.0/ + 6.2 (S25–S26);

- **Length** (90 s − 120 s): −3.1/ − 3.9 (S20–S19), +0.5/ − 0.9 (S22–S21);
- **Background** (busy − calm): +5.9/ + 7.2 (S23–S19), +14.5/ + 14.2 (S24–S21);
- **Device** (A − B, different webcams): +1.0/ + 0.1 (S25–S23), −8.4/ − 9.4 (S26–S24).

The only consistent and large effect is the **background**: a busy background raises the error by 6 to 15 percentage points with both base models. Glasses usually raise the error (three of four pairs). The length (90 vs. 120 s) shows no clear effect, and the comparison of the two devices is contradictory. Each condition appears only in single sessions. These values are therefore only descriptive; they are not statistically tested.

4.5. Strategies for Splitting the Data (E2)

The random split from Section 4.4 has a weakness: neighbouring frames of a video are almost identical (6 labels per second). The test data therefore contain almost exact copies of training frames. This is a data leak, and it makes the random split too optimistic. The following comparison measures how large this effect is.

Sessions. I use sessions 21 and 22. I choose them by a fixed rule: they are the only two sessions that differ in only one factor (length 120 s vs. 90 s). Otherwise they were recorded under the best conditions (Gen 4, no glasses, calm background, same device, same day). They are therefore suitable for the transfer between two videos, and both have enough frames (720 and 540) for temporal blocks.

Strategies.

Random (`--split random`): 70 % / 15 % / 15 % of the frames are drawn at random (reference; contains almost identical frames in training and test).

Sequential (`--split sequential`): The frames stay in recording order. The first 70 % are training, the next 15 % validation and the last 15 % test. At the start of the validation and the test block, one second (6 labels) is left out so that no neighbouring frames lie on both sides of a border. This matches real use: the user first calibrates and then uses the tracker.

Two videos (`--split two_videos`): 80 % of the frames of the first session are training, the other 20 % (random) validation. The second session is used *only* as test (all frames). I test both directions (21→22 and 22→21). This shows whether a calibration also works for a new recording of the same person.

Fair comparison. All strategies use the same base model (`norm_subject`), the same training settings, early stopping on their own validation data and the same three seeds (42, 43, 44). For each run I report the constant baseline (mean of the *training* targets, evaluated on the test frames of this run) and the error of the unchanged base model on the same test frames. This makes the different test data comparable. No test frame is used for training or model selection.

Table 4.15.: Strategies for splitting the data (sessions 21 and 22, base model `norm_subject`, original labels). Mean over three seeds; SD: standard deviation of the test error over the seeds; Base: unchanged base model; Red.: reduction of the test error relative to the constant baseline; n: training / validation / test frames; t: duration of one run (s).

Strategy	Train → test	Test (%)	SD	Val. (%)	Base (%)	Const. (%)	Red. (%)	n	t (s)
Random	21 → 21	14.40	3.13	14.15	31.52	31.25	53.9	503 / 108 / 109	3.9
	22 → 22	13.30	0.38	15.98	30.97	30.68	56.7	378 / 81 / 81	3.1
Sequential	21 → 21	14.47	1.22	19.09	27.62	27.43	47.2	503 / 102 / 103	2.0
	22 → 22	16.63	0.14	14.41	28.83	29.91	44.4	378 / 75 / 75	2.4
Two videos	21 → 22	16.79	0.13	15.67	31.05	32.17	47.8	576 / 144 / 540	5.0
	22 → 21	17.70	0.42	14.33	32.03	33.13	46.6	432 / 108 / 720	3.2
Random (mean)		13.85			31.25	30.97	55.3		
Sequential (mean)		15.55			28.22	28.67	45.8		
Two videos (mean)		17.24			31.54	32.65	47.2		

Result. The expected order holds: random (13.85 %) < sequential (15.55 %) < two videos (17.24 %). With 6 runs per strategy (two sessions × three seeds), only the difference between sequential and two videos is significant (Mann-Whitney U, $p = 0.009$). Random vs. sequential ($p = 0.09$) and random vs. two videos ($p = 0.06$) are not. Because the constant baselines of the test data differ, the relative reduction is the fairer measure: 55 % for the random split, but only 46 % to 47 % for the realistic strategies. The random split therefore overestimates the benefit of personalization by about 8 to 9 percentage points of reduction.

Interpretation.

- The random split is too optimistic, but for these two good sessions only moderately (1.7 percentage points). For difficult sessions the effect is much larger (Section 4.6.2).
- A calibration also works for a second recording of the same person under the same conditions: on a never-seen video, the model reaches 17 % to 18 % with 47 % reduction. The transfer costs about 3 percentage points compared with the random split.
- The results with two videos vary least between seeds (SD below 0.5), because the test data are large (540 to 720 frames). The random split varies most (SD 3.1 for S21), because its 81 to 109 test frames depend on the seed.

- For the sequential split, the validation error predicts the test error badly (S21: 19.1 % validation vs. 14.5 % test). Validation and test block show different parts of the screen.

For an eye tracker that is calibrated once and then used, the values of the sequential split and of the two videos are the realistic estimate.

4.6. Design of the Calibration Session

The design of the session decides which screen areas the calibration data cover. Because the errors are largest at the edges and corners (Section 4.3), the design should cover these areas well. I recorded three variants:

- **Standard video** (Gen 1, `base_Code.py`): the target moves smoothly and without jumps across the screen;
- **Random jumps** (Gen 2): the target jumps every 1.4 to 2.7 s to a random new start point;
- **Corner-weighted jumps** (Gen 4, `base_Code_rand.py`): 5-s segments with 0.5 s WAIT at a new start point and 4.5 s MOVE; 90 % of the start points lie in the four corner areas (each 10 % × 10 % of the screen).

4.6.1. Screen Coverage and Transition Frames

Table 4.16 is computed directly from the label and position files of the 18 sessions (`session_design.py`); no training is needed. The region shares use the partition from Section 4.3.1 ($r = 0.15$). Transition frames are labels recorded while the gaze is still moving to the new target: in Gen 4 the logged WAIT phase, in Gen 2 the labels up to 0.5 s after a jump (estimated, no phase logging), in Gen 1 none.

Table 4.16.: Screen coverage and transition frames per design (mean over the sessions of each generation). Cells: share of the 100 cells of a 10 × 10 grid with at least one label; Dist.: mean distance of the targets from the screen centre (normalized units).

Design	Labels	Length	Jumps	Transition	Share of labels (%)			Cells	Dist.
		(s)		(%)	Centre	Edges	Corners	(%)	
Standard (Gen 1, 4 sess.)	493	82	0	0	7.1	25.0	23.0	97	0.383
Random jumps (Gen 2, 5 sess.)	179	30	16	28.5[a]	6.3	23.0	26.3	57	0.384
Corner-weighted (Gen 4, 9 sess.)	644	107	20	10.1	6.1	19.5	36.1	91	0.420

[a] Estimated: labels up to 0.5 s after a jump.

Observations.

- **The corner weighting works as planned:** Gen 4 has 36 % of its labels near the corners, Gen 1 only 23 % and Gen 2 26 %. Gen 4 also has the largest mean distance from the centre.
- **A smooth movement covers the screen best:** Gen 1 reaches 97 % of the grid cells. The short Gen 2 recordings (30 s) cover only 57 %.
- **Transition frames:** With jumps every 1.4 to 2.7 seconds and no phase logging, on average about 28.5 % of the Gen 2 labels fall into a transition (estimated). Gen 4 has 10 %; these are logged exactly and can be removed (Section 4.4.4).

4.6.2. Personalization Result (E3)

The random split favours designs with slow, smooth movement, because almost identical neighbouring frames end up in training and test. I therefore also trained all 18 sessions with the sequential split (Section 4.5; base model `norm_subject`, seed 42). Table 4.17 compares both splits per design. Table 4.18 shows the errors per screen region, computed from the stored test predictions.

Table 4.17.: Test error per design with random and sequential split (mean over the sessions; base model `norm_subject`). Const.: constant baseline of the respective test frames; Ep. 0: number of sessions with best epoch 0 (no improvement over the base model); n_{test}: mean number of test frames.

	Random split		Sequential split			
Design	Test	Const.	Test	Const.	Ep. 0	n_{test}
Standard (Gen 1)	15.29	29.22	21.52	28.70	1 / 4	69
Random jumps (Gen 2)	23.45	29.44	32.80	34.39	2 / 5	22
Corner-weighted (Gen 4)	19.76	31.31	28.07	31.82	5 / 9	91
All (18)	19.79	30.32	27.93	31.84[a]	8 / 18	67

[a] Mean of the session values of the sequential split.

Sequential split. Without almost identical frames in training and test, the error rises for every design, on average from 19.8 % to 27.9 % (Wilcoxon $p = 4 \cdot 10^{-5}$). The personalized model is still better than the constant baseline (Wilcoxon $p = 0.002$), but the reduction shrinks from 35 % to about 12 %. In 8 of 18 sessions (02, 05, 08, 23–27), the best epoch is 0: training on the first 70 % of the recording does not improve the error on the validation block. The reported result is then the unchanged base model. This happens mainly in Gen 4 with a busy background (23–27) and in the short Gen 2 recordings, whose validation and test blocks have only 17 to 27 frames. The random split from Section 4.4 therefore strongly overestimates the benefit of personalization for difficult sessions.

Table 4.18.: Test error per screen region (Section 4.3.1, $r = 0.15$), mean over the sessions of a design; in brackets the total number of test frames in the region. Base model `norm_subject`.

	Random split				Sequential split			
Design	All	Centre	Edges	Corners	All	Centre	Edges	Corners
Standard (Gen 1)	15.3	17.7 (20)	13.5 (67)	16.7 (83)	21.5	11.9 (25)	20.9 (64)	28.9 (52)
Random jumps (Gen 2)	23.4	13.7 (12)	21.6 (34)	26.1 (42)	32.8	6.6 (2)[a]	29.8 (15)	38.6 (45)
Corner-weighted (Gen 4)	19.8	11.5 (45)	14.9 (175)	26.5 (299)	28.1	9.0 (60)	23.9 (136)	37.1 (289)
All (18)	19.8	13.5 (77)	16.4 (276)	24.2 (424)	27.9	9.7 (87)	24.3 (215)	36.1 (386)

[a] Only one session with two frames in the centre; not meaningful.

Designs. For both splits, the standard video (Gen 1) has the lowest and the random jumps (Gen 2) the highest error. Even with the sequential split, the differences are not significant (Kruskal-Wallis $p = 0.06$). The order can be explained by the coverage (Table 4.16): Gen 1 covers 97 % of the screen in every part of the recording. For the jump designs, a temporal block only covers the targets that were shown during that time. The short Gen 2 recordings suffer most from this. The designs are also mixed with light, background and glasses. So the order is a trend, not a proof.

Regions. The corners are the hardest region for every design and both splits (24 % random, 36 % sequential), the centre the easiest (10 to 14 %). The corner-weighted design does *not* lower the corner error: Gen 4 has 26.5 % / 37.1 % in the corners, the standard video 16.7 % / 28.9 %. More corner labels alone do not help when the gaze in the corners is harder to estimate (head rotation, half-closed eyes, camera position) and when the corner labels were recorded under worse conditions (Gen 4 sessions 23–27). In addition, the sigmoid output has to be close to 0 or 1 in the outer corners.

Consequence for the design. With the available data, a smooth target movement across the whole screen gives the best result for a short calibration. Jumps create transition frames (Gen 2: on average about 28.5 % of the labels). If jumps are used, they must be logged so that these frames can be removed. The corner weighting raises the share of corner labels as planned, but it does not lower the corner error.

4.7. Discussion of the Results

Main result. On GazeCapture, all settings of the hyperparameter study reach practically only the error of a constant estimator on the same test data: the best settings with "FC only" lie between 28.32 % and 28.39 %, the constant baseline at 28.46 %. The spatial

analysis confirms that the predictions stay in the screen centre. The hyperparameter study therefore separates the settings mainly by their *stability*: options that make training less stable (Gaussian, ReLU, no activation, more trainable layers, sensitive loss) are worse. But no option clearly leaves the centre. The decisions in Section 4.2 are therefore based on stability, learning behaviour, effort and methodological reasons, as described in each subsection. Single notebook runs on `norm_random` (Figures 4.1b and 4.2) did reach about 23.6 % with a constant baseline of 26.8 % on their test data. These runs, however, used an older data selection. With the current protocol (random data selection, five runs), such a learning effect is not visible. The older values are therefore no evidence for a learning effect. The base models confirm this: their error on the personal sessions (30.6 %) equals the constant baseline.

Possible causes. The new protocol already rules out two explanations: the training data now contain 179 instead of five persons, and the test data are no longer used for model selection. Still, the predictions stay in the centre. The most likely causes are: (i) I use whole camera images. The eyes are only a small part of the image. The frozen ImageNet features therefore contain little gaze information. (ii) 1000 frames of 179 persons are only about six frames per person. This is too few to learn a mapping for all persons. More trainable layers with so few data only make training unstable (Section 4.2.3). Face or eye crops as input and more training data are the next steps to test.

Personalization. With the same frozen backbone, a head trained on a few hundred frames of one person lowers the error compared with the constant baseline. With a temporal split, which matches real use, the reduction is about 12 % on average (31.8 % $\rightarrow$ 27.9 %) and zero in difficult sessions. Good sessions, however, keep most of the gain, and a calibration even works for a second recording (47 % reduction, Section 4.5). The random split shows a reduction of about one third, but it is too optimistic. One run takes about two seconds on a CPU, and the choice of base model does not matter; it mainly provides a starting point. This supports the central idea of this thesis, data-efficient personalization of a pretrained model. In addition, the recording conditions (background, light, glasses) affect the result more than the design of the session.

Changes to the code. The error metric was corrected. Baselines are computed on the same subset as the model. `exp.py` has new options for random subsets (`--sample-seed`) and a subject-disjoint validation set (`--val-size`). Full fine-tuning (`fc_resnet`) can be included with `--include-resnet`. The personalization runs are reproducible (fixed seeds, frozen BatchNorm, feature cache, stored predictions, `personal_exp.py`).

4.8. Summary of the Results

Table 4.19 summarizes all experiments. "Significant" means $p < 0.05$ in the test named in the respective section. All errors are diagonal errors (Equation (4.1)) in %. MAE and MSE were not logged in these experiments. The diagonal error (a scaled Euclidean RMSE) is therefore the only accuracy measure that is available and comparable for all experiments.

Table 4.19.: Summary of the results and the chosen setting. GazeCapture: `norm_subject`, 1000/200 random frames, five runs, constant baseline 28.46 %; time = mean duration of a training run in minutes (head on stored features). Personalization: mean over the sessions, time = one run per session in seconds (Section 4.4.3).

Experiment	Chosen	Error (%)	Time	Result and reason
Baselines	—	28.46	—	constant baseline on the same test data; random baseline 40.36 (Section 4.1.5)
Target format	normalized $[0, 1]^2$	—	—	independent of the device; metric well-defined (Section 4.1.4)
Batch size	32	28.39	0.019 min	no significant difference (Section 4.2.1)
Learning rate	10^{-4}	28.37	0.020 min	lowest error in the first test, fast; repetition without significant difference (Section 4.2.2)
Trainable layers	FC only	28.36	0.015 min	lowest and most stable error; FC + layer 4: 9.2 min, full: 15.4 min (Section 4.2.3)
Activation	Sigmoid	**28.36**	0.022 min	**significantly** better than Gaussian, ReLU, None (Section 4.2.4)
Loss function	L1	**28.32**	0.011 min	**significantly** better than the sensitive loss (Section 4.3.4)
Amount of data	1000/200	28.39	0.018 min	larger set only better because of other test data (Section 4.2.5)
Split	`norm_subject`	28.39	0.020 min	only split that measures unknown persons (Section 4.2.6)
Personalization (E1)	FC only, 18 sessions	**19.79**	1.9 s	**significantly** better than constant baseline (30.32) and base model (30.57), −35 % (Section 4.4.2)
Base model (E0/E1)	`norm_subject`	19.79	1.9 s	same as `norm_random` (19.59; 2.4 s; $p = 0.20$); chosen because it contains no information about the test persons
Cleaning (E1b)	remove WAIT frames	15.88	—	vs. 16.72 on the same MOVE test frames; −0.84, not significant; WAIT frames themselves 37.7 (Section 4.4.4)
Data split (E2)	sequential / two videos	15.55 / 17.24	2–5 s	random 13.85 is too optimistic; two videos significantly worse than sequential; calibration also works for a new video (−47 %) (Section 4.5)
Session design (E3)	standard video	15.29 / 21.52	—	random / sequential split; random jumps 23.45 / 32.80, corner-weighted 19.76 / 28.07; not significant and mixed; corner weighting does not lower the corner error (Section 4.6.2)

5. Discussion

5.1. Answers to the Research Questions

RQ1: How much does a short personalization help? The generic base model is not better than the constant baseline on the own sessions (30.57 % compared with 30.32 %). With a temporal split, which matches real use, personalization lowers the error from 31.84 % to 27.93 %, that is, by about 12 % (Wilcoxon $p = 0.002$). In 8 of 18 sessions there is no improvement (Section 4.6.2). In the good sessions 21 and 22, however, the gain stays large (46 %) and even holds for a second recording (47 % reduction, Section 4.5). With a random split, the error drops to 19.79 %, that is, by about one third, and the best session reaches 10.5 % (Section 4.4.2). This split, however, is too optimistic. *Answer:* a short personalization helps clearly, but only under good recording conditions. The random split overestimates the benefit.

RQ2: Which training setting is suitable? On GazeCapture, most settings do not differ significantly, because all models stay close to the constant baseline (Section 4.2). It is clear that sigmoid is the best activation and that L1 is better than the loss weighted by position. Training only the head ("FC only") is as good as more layers, but more stable and much faster. *Answer:* frozen backbone, regression head with sigmoid, L1 loss, learning rate 10^{-4} and batch size 32. The choice is based mainly on stability and effort; for the learning rate also on the lowest error in the first test.

RQ3: What is the role of data split and calibration? The split is very important. The random split is too optimistic, because almost identical neighbouring frames end up in training and test. The smooth movement of the target (Gen 1) covers the screen best and had the lowest error. Jumps create frames with wrong labels. Removing these WAIT frames improves the result slightly (0.84 percentage points, not significant). More labels in the corners do not lower the corner error. *Answer:* a short, smooth movement across the whole screen is best. Jumps should be logged so that transition frames can be removed. Evaluation should always use a temporal split.

RQ4: What is the influence of the recording conditions? The strongest factor is the background: a busy background raises the error by 6 to 15 percentage points. Glasses usually raise it, and in dark rooms personalization hardly helps. Length and device (webcam) show no clear effect (Section 4.4.5). *Answer:* a calm background and good light are more important than the design of the calibration. These statements, however, are based on single sessions.

5.2. Relation to the State of the Art

Some results fit the literature well:

- **Few parameters are enough.** As in offset calibration [27], SPAZE [28] and PIGaze [39], a small adapted head on a frozen backbone is enough here. More trainable layers only make training less stable. This fits Kumar et al. [22].
- **Personalization is the most important factor.** As in Valliappan et al. [7], almost the whole gain comes from a few seconds of personal data.
- **The evaluation protocol decides.** Similar to the difference between `norm_random` and `norm_subject`, random splits overestimate the performance (Section 2.7.1).
- **Wrong labels have a small effect.** DMRF [41] also lowers the error only a little ($3.92° \rightarrow 3.85°$). Here, removing the WAIT frames gives 0.84 percentage points; this difference is not significant ($p = 0.43$).

There is also an important difference: the generic model of this thesis is very weak. It stays at the screen centre, while methods in the literature reach a few degrees of accuracy even without personalization. The main reason is probably the input: most methods use face or eye crops, while this thesis uses the whole camera image. In addition, the base model uses only 1000 frames of 179 persons. The absolute numbers are in any case not directly comparable with the literature because of the different metrics (Section 2.10).

5.3. Limitations

- **One person:** all own recordings come from one person (the author), recorded with two devices. The results therefore do not automatically hold for other persons.
- **Few sessions and mixed conditions:** 18 sessions are not enough for reliable statements about single factors. Design, light, background and glasses often change at the same time.

- **Single runs:** there is usually only one run per session (seed 42). The spread between seeds can be up to 3 percentage points.
- **One backbone and whole images:** only ResNet-18 with whole camera images was studied. DINOv2, CLIP, MAE, UniGaze, adapters and LoRA were not tested, and neither were face or eye crops in the experiments of Chapter 4.
- **Weak base model:** the base model hardly learns anything on GazeCapture and mainly serves as a starting point for personalization in this thesis. Whether a randomly initialized head would work equally well was not tested. A better base model could change the personalization results.
- **Many tests:** many statistical tests were computed without a correction for multiple comparisons (for example Bonferroni). Close results such as $p = 0.032$ (L1 versus the sensitive loss) should therefore be read with care.
- **Metric:** the diagonal error is not directly comparable with angular errors or centimetres from the literature.

6. Conclusion and Outlook

6.1. Conclusion

This thesis studied how well a pretrained vision model can be adapted to one person with a short webcam calibration. A ResNet-18 pretrained on ImageNet stayed frozen, and only a small regression head was trained.

The most important results are:

- A generic model trained on GazeCapture is not better than an estimate that always gives the screen centre. It therefore mainly serves as a starting point for personalization.
- With a realistic, temporal split, a few minutes of personal data lower the error from 31.8 % to 27.9 % of the screen diagonal (about −12 %, 18 sessions). In good sessions the gain stays large (about −46 %) and also holds for a new recording of the same person. One run takes about two seconds on a CPU.
- A random split gives 19.8 % instead of 30.3 % (about −35 %). It overestimates the benefit, however, because almost identical neighbouring frames end up in training and test.
- A calm background and good light are more important than the design of the calibration. A smooth movement of the target across the whole screen was best.
- Frames directly after a jump of the target have wrong labels. Removing them helps a little, but costs nothing.

Overall, the thesis shows: data-efficient personalization works with simple means. But the accuracy depends strongly on the recording conditions, and it must be assessed with a realistic data split. All recordings come from one person (the author). The results therefore hold for this person only at first; whether they transfer to other persons must be shown by a study with several persons.

6.2. Outlook

Several next steps are obvious:

- **Eye or face crops as input.** In the whole camera image, the eyes are very small. A crop should give the model much more gaze information.
- **Other pretrained models.** DINOv2, CLIP, MAE or UniGaze instead of ResNet-18, each with a frozen backbone and a small head.
- **Lightweight adaptation of the backbone.** Adapters or LoRA as a middle way between head-only training and full fine-tuning.
- **More persons and more sessions,** with conditions that change in only one factor at a time.
- **Taking head movement into account,** for example with the head position as an extra feature or with normalized face images.

Web demo. As a practical test, I built a demo in the browser (server with FastAPI, suitable for Hugging Face Spaces). The user follows a point for 60 seconds. Then a model is personalized, and the gaze is shown live as a light field on a grid of 3×3 cells. At the end, an evaluation shows when which cell was looked at. The demo already uses some ideas from the outlook: an eye crop, spatial features of the ResNet-18, a ridge regression and the head position as an extra feature. A short test with five points (3 s each) after the calibration measures the error on new frames. In several single tests, this error was between about 17.6 % and 25.2 % (last test: 19.0 %); a constant estimate reaches about 23 % on the calibration data. In a direct comparison, the test error fell from 23.0 % to 17.6 % with the head position. The values vary strongly between runs and were not evaluated systematically. But they show that head movement and light are large sources of error in live use.

References

[1] Kyle Krafka, Aditya Khosla, Petr Kellnhofer, Harini Kannan, Suchendra Bhandarkar, Wojciech Matusik, and Antonio Torralba. Eye tracking for everyone. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2176–2184, 2016. doi: 10.1109/CVPR.2016.239.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[3] Yihua Cheng, Haofei Wang, Yiwei Bao, and Feng Lu. Appearance-based gaze estimation with deep learning: A review and benchmark. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):7509–7528, 2024. doi: 10.1109/TPAMI.2024.3393571.

[4] Jiahui Liu, Jiannan Chi, Huijie Yang, and Xucheng Yin. In the eye of the beholder: A survey of gaze tracking techniques. *Pattern Recognition*, 132:108944, 2022. doi: 10.1016/j.patcog.2022.108944.

[5] Primesh Pathirana, Shashimal Senarath, Dulani Meedeniya, and Sampath Jayarathna. Eye gaze estimation: A survey on deep learning-based approaches. *Expert Systems with Applications*, 199:116894, 2022. doi: 10.1016/j.eswa.2022.116894.

[6] Alexandra Papoutsaki, Patsorn Sangkloy, James Laskey, Nediyana Daskalova, Jeff Huang, and James Hays. Webgazer: Scalable webcam eye tracking using user interactions. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3839–3845, 2016.

[7] Nachiappan Valliappan, Na Dai, Ethan Steinberg, Junfeng He, Kantwon Rogers, Venky Ramachandran, Pingmei Xu, Mina Shojaeizadeh, Li Guo, Kai Kohlhoff, and Vidhya Navalpakkam. Accelerating eye movement research via accurate and affordable smartphone eye tracking. *Nature Communications*, 11:4553, 2020. doi: 10.1038/s41467-020-18360-5.

[8] Xucong Zhang, Yusuke Sugano, Mario Fritz, and Andreas Bulling. Appearance-based gaze estimation in the wild. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4511–4520, 2015.

[9] Xucong Zhang, Yusuke Sugano, Mario Fritz, and Andreas Bulling. Mpiigaze: Real-world dataset and deep appearance-based gaze estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(1):162–175, 2019. doi: 10.1109/TPAMI.2017.2778103.

[10] Xucong Zhang, Seonwook Park, Thabo Beeler, Derek Bradley, Siyu Tang, and Otmar Hilliges. Eth-xgaze: A large scale dataset for gaze estimation under extreme head pose and gaze variation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. doi: 10.1007/978-3-030-58558-7_22.

[11] Seonwook Park, Emre Aksan, Xucong Zhang, and Otmar Hilliges. Towards end-to-end video-based eye-tracking. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. doi: 10.1007/978-3-030-58610-2_44.

[12] Petr Kellnhofer, Adrià Recasens, Simon Stent, Wojciech Matusik, and Antonio Torralba. Gaze360: Physically unconstrained gaze estimation in the wild. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6911–6920, 2019.

[13] Seonwook Park, Shalini De Mello, Pavlo Molchanov, Umar Iqbal, Otmar Hilliges, and Jan Kautz. Few-shot adaptive gaze estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9368–9377, 2019. doi: 10.1109/ICCV.2019.00946.

[14] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[15] Jiawei Qin, Xucong Zhang, and Yusuke Sugano. Unigaze: Towards universal gaze estimation via large-scale pre-training. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 5809–5820, 2026.

[16] Vikrant Nagpure, Tanisha Jain, Rongali Sai Bhargav, Ashwin K. Krishna, and Kenji Okuma. Learning to adapt: Test-time personalized gaze estimation with weak and self-supervised learning for in-vehicle scenarios. In *Proceedings of the IEEE Intelligent Vehicles Symposium (IV)*, pages 1334–1339, 2025. doi: 10.1109/IV64158.2025.11097756.

[17] Jun Wang, Hao Ruan, Liangjian Wen, Yong Dai, and Mingjie Wang. Gazeclip: Enhancing gaze estimation through text-guided multimodal learning. *Neurocomputing*, 685:133584, 2026. doi: 10.1016/j.neucom.2026.133584.

[18] Lang Ye, Xinggang Wang, Jingfeng Yao, and Wenyu Liu. Transgaze: exploring plain vision transformers for gaze estimation. *Machine Vision and Applications*, 35:128, 2024. doi: 10.1007/s00138-024-01609-0.

[19] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.

[20] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, 2021.

[21] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022.

[22] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[23] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[24] Xucong Zhang, Yusuke Sugano, and Andreas Bulling. Revisiting data normalization for appearance-based gaze estimation. In *Proceedings of the ACM Symposium on Eye Tracking Research & Applications (ETRA)*, 2018.

[25] Pengwei Yin, Guanzhong Zeng, Jingjing Wang, and Di Xie. Clip-gaze: Towards general gaze estimation via visual-linguistic model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 6729–6737, 2024. doi: 10.1609/aaai.v38i7.28496.

[26] Junfeng He, Khoi Pham, Nachiappan Valliappan, Pingmei Xu, Chase Roberts, Dmitry Lagun, and Vidhya Navalpakkam. On-device few-shot personalization for real-time gaze estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2019.

[27] Zhaokang Chen and Bertram E. Shi. Offset calibration for appearance-based gaze estimation via gaze decomposition. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020.

[28] Erik Lindén, Jonas Sjöstrand, and Alexandre Proutière. Learning to personalize in appearance-based gaze tracking. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 1140–1148, 2019. doi: 10.1109/ICCVW.2019.00145.

[29] Gang Liu, Yu Yu, Kenneth A. Funes Mora, and Jean-Marc Odobez. A differential approach for gaze estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(3):1092–1098, 2021. doi: 10.1109/TPAMI.2019.2957373.

[30] Xiaoyu Zhao, Yaping Huang, Yi Tian, and Mei Tian. Episode-based personalization network for gaze estimation without calibration. *Neurocomputing*, 513:36–45, 2022.

[31] Yu Yu, Gang Liu, and Jean-Marc Odobez. Improving few-shot user-specific gaze adaptation via gaze redirection synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[32] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799, 2019.

[33] Huan Liu, Julia Qi, Zhenhao Li, Mohammad Hassanpour, Yang Wang, Konstantinos N. Plataniotis, and Yuanhao Yu. Test-time personalization with meta prompt for gaze estimation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 3621–3629, 2024. doi: 10.1609/aaai.v38i4.28151.

[34] Yiwei Bao and Feng Lu. From feature to gaze: A generalizable replacement of linear layer for gaze estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1409–1418, 2024. doi: 10.1109/CVPR52733.2024.00140.

[35] Yaoming Wang, Yangzhou Jiang, Jin Li, Bingbing Ni, Wenrui Dai, Chenglin Li, Hongkai Xiong, and Teng Li. Contrastive regression for domain adaptation on gaze estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19376–19385, 2022. doi: 10.1109/CVPR52688.2022.01877.

[36] Yihua Cheng, Yiwei Bao, and Feng Lu. Puregaze: Purifying gaze feature for generalizable gaze estimation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.

[37] Mingfang Zhang, Yunfei Liu, and Feng Lu. Gazeonce: Real-time multi-person gaze estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4197–4206, 2022. doi: 10.1109/CVPR52688.2022.00416.

[38] Yong Wu, Yang Wang, Sanqing Qu, Zhijun Li, and Guang Chen. Elf-ua: Efficient label-free user adaptation in gaze estimation. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

[39] Yafei Wang, Haiheng Nan, Runze Yan, Xueyan Ding, Jianda Wang, and Xianping Fu. Pigaze: Personalized in-vehicle gaze estimation with plug-and-play adaptation. *Knowledge-Based Systems*, 348:116336, 2026.

[40] Diederick C. Niehorster, Marcus Nyström, Roy S. Hessels, Jeroen S. Benjamins, Richard Andersson, and Ignace T. C. Hooge. The fundamentals of eye tracking, part 7: Determining data quality. *Behavior Research Methods*, 58:183, 2026. doi: 10.3758/s13428-026-03039-4.

[41] Gaoxia Jiang, Fan Lei, and Wenjian Wang. A multi-model dynamic filtering of label noise for regression. *Pattern Recognition*, 169:111887, 2026. doi: 10.1016/j.patcog.2025.111887.

[42] Yanming Peng, Shijing Wang, Yaping Huang, and Yi Tian. See through the noise: Improving domain generalization in gaze estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 31346–31355, 2026.

[43] dhruv413. GazeCapture 20 GB (ZIP). Kaggle. URL `https://www.kaggle.com/datasets/dhruv413/gaze-capture-20gb-zip`. Subset of the GazeCapture dataset.

A. Appendix

A.1. Commands to Repeat the Experiments

All commands are run in the main folder of the project (see also `README.md`).

Prepare the data.

```
python main.py
```

Baselines and hyperparameter study (GazeCapture).

```
python exp.py --experiment baseline
python exp.py --experiment all --sample-seed 0 --val-size 200
python exp.py --experiment trainable_layers --include-resnet
```

Spatial analysis and figures.

```
python spatial_analysis.py --input results/heatmap/base_loss1.csv \
       --compare results/heatmap/base_loss2.csv --out results/figures
python plot_targets.py --out results/figures
```

Personalization (E0–E3).

```
python train_base.py
python personal_exp.py --sessions all --base norm_subject norm_random
python personal_exp.py --sessions gen4 --labels clean
python personal_exp.py --sessions 21 22 --split random --seeds 42 43 44
python personal_exp.py --sessions 21 22 --split sequential --seeds 42 43 44
python personal_exp.py --split two_videos --pairs 21:22,22:21 --seeds 42 43 44
python personal_exp.py --sessions all --split sequential
python personal_exp.py --summary
```

Session design and cleaned labels.

```
python session_design.py
python session_design.py --write-clean
```

It is easier to use the notebook `notebooks/Masterarbeit.ipynb`: cell 1 trains the base models (E0), cell 3 runs the personalization (E1 to E3) and creates all tables.

Statutory Declaration

I hereby declare that I have written this master's thesis independently and have used no sources or aids other than those stated, that all passages taken literally or in substance from other writings are marked as such, and that the thesis has not been part of any other course or examination in the same or a similar form.

Use of AI Tools

While preparing this thesis, I used the following AI tools as aids:

Tool	**Purpose**	**Affected parts**
ChatGPT (OpenAI)	Testing the implementation; improving the structure of the program code	Program code
Claude (Anthropic)	**Code:** reviewing and evaluating the code, finding and correcting errors, scripts for recomputing the results (`auswertung.py`, notebook), improving the web demo	Program code, tables and figures in Chapter 4
Claude (Anthropic)	**Text:** translation between German and English, language editing and simplification, writing out text passages based on my drafts, results and notes, correcting errors	All chapters, abstract
Claude (Anthropic)	**Layout:** LaTeX design (title page following the BHT template), labelling of figures	Layout, figures

The topic, the research questions, the design of the experiments, the recording of the webcam data and the training runs are my own work. I checked all results of the AI tools critically and corrected them where necessary; the numbers in Chapter 4 were recomputed from the stored result files. The technical statements are based on the cited literature; AI tools are not used as sources. I take full responsibility for the entire content of this thesis.

Place, date

Khaled Rafei